ZOHO

https://gemini.google.com/app/d10b3546cd922eed

User prompt: I want to make an alternative to Zoho Project Management System. So basically, it is a team-based project management system where we can create and manage product projects and tasks, task list sheets, etc. So can you please gather information about Zoho Projects or all of the equivalent project lines that are there in the market? And let's please start building a file repository system. How many files would be required there to build this specific project? Right now, I just need an extensive and detailed research of the number of features, the number of available screens or the things that are required for this. Then we need the file structure, how the files will be made, and then we will discuss about the after things. So for now, please don't care about the context length or the token limit. Just please search and refer to multiple sources and gather these things first of all.

Response: Building a full-scale alternative to **Zoho Projects** requires combining a **structured Work Breakdown Structure (WBS)** with the speed and multi-view flexibility of modern tools like **ClickUp, Linear, Asana, and Plane.**

Below is the complete market analysis, an exhaustive breakdown of **142 features across 12 core modules**, an inventory of **54 distinct screens and UI surfaces**, realistic **file count benchmarks**, and a **production-grade Monorepo file structure** showing how every layer is constructed.

1. Market Landscape & Architectural Benchmark

Before designing the repository, it is essential to understand how Zoho Projects compares to other leading project management architectures in the market (both commercial SaaS and open-source benchmarks):

Platform	Core Architectural Strength	Hierarchy / Data Model	Key Differentiator to Adopt
Zoho Projects	Strict Work Breakdown Structure (WBS), Process Automation & Financials	`Portal -> Project Group -> Project -> Phase (Milestone) -> Task List -> Task -> Subtask` (+ `Issues`)	**Task List Sheets**, **Blueprints** (visual state-machine workflows), **EVM Budgeting**, and internal vs. external client flags.
ClickUp	Ultra-flexible multi-view engine & custom metadata	`Workspace -> Space -> Folder -> List -> Task -> Subtask -> Checklist`	Dynamic **Custom Fields**, 15+ switchable views on the same dataset, and embedded Docs/Whiteboards.
Linear	High-speed keyboard-first UX & local-first sync engine	`Workspace -> Team -> Initiative/Project -> Cycle -> Issue -> Sub-issue`	Instant **IndexedDB/GraphQL sync engine**, Command Palette (`Cmd+K`), and zero-latency inline editing.
Asana	Cross-team dependency graph & Goal/Portfolio alignment	`Organization -> Team -> Portfolio -> Project -> Section -> Task -> Subtask`	Multi-homing (one task living in multiple projects simultaneously) and **Critical Path** calculation.
Jira	Issue tracking, SLA escalation & Agile sprint velocity	`Site -> Project -> Epic -> Story/Task/Bug -> Sub-task`	Granular permission schemes, custom issue screens, and deep Git/CI-CD linkage.
Smartsheet	Spreadsheet-native project & resource management	`Workspace -> Folder -> Sheet -> Row -> Child Row`	Formula columns, cell-level keyboard navigation, and cross-sheet rollups (**Task List Sheet** experience).
Plane / **OpenProject** *(Open-Source)*	Clean modular monorepo split (Web + API + Realtime + Worker)	`Workspace -> Project -> Module/Phase -> Cycle -> Work Item`	Triple-format rich text storage (JSON + HTML + Yjs Binary) and PostgreSQL advisory locks for sequential task IDs (e.g., `PRJ-104`).

The "Zoho Projects Core DNA" You Must Replicate

Unlike simple Kanban tools (like Trello), Zoho Projects revolves around **four structural pillars** that must be baked into your database schema from Day 1:

1. **The 5-Level WBS Hierarchy:** `Portal` (Tenant) -> `Project` -> `Phase / Milestone` -> `Task List` -> `Task` -> `Subtask` (recursive depth).
2. **Dual Work-Item Engine: Tasks** (scheduled work with start/end dates, durations, work allocation, and task list sheets) and **Issues / Bugs** (SLA-driven defects with severity, reproducibility, classification, and affected/release phases).
3. **Internal vs. External (Client) Scoping:** Every Phase, Task List, Task, Comment, and Document has a visibility flag (`internal` vs. `external`) so agencies and product teams can invite external clients into a portal without exposing internal discussions or cost rates.
4. **Task List Sheets & Multi-View Sync:** The ability to manipulate tasks inside a high-density, spreadsheet-like **Task List Sheet** (inline editing of dates, owners, % completion, custom fields, and formulas) that immediately reflects in Kanban, Gantt, and Calendar views.

2. Exhaustive Feature Research (142 Features Across 12 Modules)

To build a competitive Zoho Projects alternative, the platform breaks down into **12 functional modules** comprising **142 distinct features**:

Module 1: Multi-Tenant Portal, Identity & Access Control (12 Features)

1. **Multi-Portal Architecture:** Users can belong to multiple isolated company portals with unique slugs (`[app.domain.com/](https://app.domain.com/){portal_slug}`).
2. **Authentication Suite:** Email/Password, Magic Links, OAuth2 (Google, GitHub, Microsoft), and Enterprise SAML/OIDC SSO.
3. **Two-Factor Authentication (2FA) & Session Management:** TOTP authenticator support and active device revocation.
4. **Role-Based Access Control (RBAC):** Predefined and custom portal roles (Admin, Manager, Employee, Contractor).
5. **Profile & Security Policies:** Field-level read/write permissions (e.g., hiding hourly cost rates from standard members).
6. **Client Users & Client Companies:** Dedicated external user accounts restricted to `external`-flagged task lists, phases, and issues.
7. **Teams / User Groups:** Grouping users by department (Frontend, QA, Design, Marketing) for bulk assignment and notifications.
8. **Business Hours & Holiday Calendars:** Portal-wide working days, shift hours, and regional holiday schedules (used by the Gantt & work-allocation engine).
9. **Leave & Availability Tracker:** Recording employee time-off/vacation so task scheduling warns when an assignee is unavailable.
10. **Portal Custom Branding:** Custom domain mapping, light/dark/contrast themes, and portal logos.
11. **Audit Logs & Security Trail:** Immutable logs of every login, permission change, deletion, and export.
12. **Sandbox / Environment Mode:** Testing layout changes and automation blueprints before applying to production projects.

Module 2: Project Portfolio & Workspace Configuration (12 Features)

13. **Project Groups / Portfolios:** Organizing related projects by product line, department, or client.
14. **Project Creation & Lifecycle:** Active, On-Hold, Completed, Archived, and Trashed states.
15. **Project Templates:** Saving an existing project's phases, task lists, tasks, relative date offsets, layouts, and roles as a reusable blueprint.
16. **Roll-Up Projects:** Aggregating progress, budgets, and timelines from sub-projects into a master program view.
17. **Public vs. Private Projects:** Public projects visible to all portal members vs. strict invite-only projects.
18. **Strict Project Settings:** Enforcing that tasks and task lists *must* fall within the start/end dates of their parent Phase and Project.
19. **Task Prefixing & Auto-Sequencing:** Custom project key prefixes (e.g., `WEB-101`, `BUG-42`) using atomic sequence counters.
20. **Module Toggle per Project:** Enabling/disabling specific tabs (Tasks, Issues, Phases, Timesheets, Docs, Forums, Finance) per project.
21. **Custom Layouts per Project:** Associating different sets of custom fields and statuses to different project types (e.g., Software Dev vs. Marketing Launch).
22. **Project Tags & Color Coding:** Categorizing projects with global or project-scoped tags.
23. **Project Health Score & Status Updates:** Automated red/amber/green health indicators based on overdue tasks and budget burn.
24. **Cross-Project Cloning (`copy_from`):** Deep-cloning selected entities (phases, task lists, custom fields, members) into a new project.

Module 3: Work Breakdown Structure — Phases & Task Lists (10 Features)

25. **Phases (Milestones):** Time-boxed project stages with start date, end date, owner, budget, and completion progression.
26. **Sequential Phase Chaining:** Setting a phase to `start_after` a previous phase by N days/weeks/months.
27. **Internal vs. External Phases:** Controlling whether client users can see a milestone.
28. **Task Lists:** Grouping tasks under a specific Phase (or in an unassigned "General" task list).
29. **Default Task Lists:** Auto-created internal and external task lists when tasks are added without specifying a list.
30. **Task List Sequence & Drag-and-Drop Reordering:** Independent ordering within a Phase (`milestone_sequence`) and across the Project (`project_sequence`).
31. **Task List Rollover:** Automatically rolling uncompleted tasks from a closed task list/phase into the next active phase.
32. **Task List Progress Bar:** Real-time calculation of closed vs. open tasks and weighted completion percentage.
33. **Task List Comments & Attachments:** Discussions scoped specifically to a task list deliverable.
34. **Convert Task List to Template:** Exporting or saving a single task list for reuse across projects.

Module 4: Core Task & Subtask Engine (18 Features)

35. **Rich Task Creation:** Title, 64KB HTML/Markdown/Block description, priority (`none`, `low`, `medium`, `high`), status, start date, and end date.

36. **Multi-Level Subtask Hierarchy:** Recursive `parent_task_id` and `root_task_id` supporting nested subtasks (`depth` 1 to N).
37. **Completion Percentage Rollup:** Automatic or manual `0–100%` slider where parent tasks derive progress from subtasks and checklists.
38. **Multi-Owner Assignment (`owners_and_work`):** Assigning multiple team members to a single task with individual work allocations.
39. **Work Allocation Modes**:
    - *Standard Work:* Total hours, hours per day (`HH:MM`), or `%_per_day` (`0–100%`) per assignee.
    - *Flexible Work:* Day-by-day custom hour/percentage matrix for each assignee.
40. **Duration Engine:** Task duration in `days` or `hours`, automatically respecting portal business hours and holidays.
41. **Task Dependencies (4 Types):** Finish-to-Start (FS), Start-to-Start (SS), Finish-to-Finish (FF), and Start-to-Finish (SF) with lag/lead days.
42. **Auto-Rescheduling Cascade:** Shifting dependent successor tasks automatically when a predecessor task's end date slips.
43. **Recurring Tasks:** Daily, weekly, monthly, or yearly recurrence with interval rules, `next_recurrence_after_close`, `set_to_previous_businessday`, and optional subtask/comment retention.
44. **Smart Task Reminders:** Reminders triggered `daily`, `on` a custom date, or `before`/`after` start/end dates, notifying specific users, teams, or reporting managers.
45. **In-Task Checklists (To-Do Items):** Lightweight checkable sub-items inside a task card.
46. **Task Billing Classification:** Marking tasks as `billable`, `non_billable`, or `none`.
47. **Followers / Watchers:** Subscribing non-assignees to receive status and comment notifications.
48. **Task Voting & Reactions:** Upvoting tasks in backlog prioritization and emoji reactions.
49. **Task Linking & Relations:** `relates_to`, `duplicates`, `blocks`, `is_blocked_by` cross-project references.
50. **Bulk Task Operations:** Multi-select toolbar to bulk-update status, owner, priority, dates, tags, or move between task lists.
51. **Trash & Soft-Delete Recovery:** 30-day recycle bin for accidentally deleted tasks, lists, or phases.
52. **Keyboard Shortcuts & Command Palette:** `C` to create task, `Q` for quick cycle/phase, `Cmd+K` for global navigation.

Module 5: Task List Sheets & Multi-View Visualization Engine (11 Features)

53. **Task List Sheet View (Spreadsheet Grid):** High-density virtualized table grouped by Phase and Task List, with keyboard arrow navigation, inline cell editing, column resizing, frozen columns, and sum/average footer rows.
54. **Kanban Board View:** Drag-and-drop cards grouped by Status, Priority, Assignee, Phase, or Custom Picklist, with WIP (Work-in-Progress) column limits and horizontal swimlanes.
55. **Interactive Gantt Chart View:** Timeline bars for Phases, Task Lists, Tasks, and Subtasks with drag-to-resize dates, dependency connector arrows, **Critical Path** highlighting, and **Baseline** comparison (Planned vs. Actual slip).
56. **Calendar View:** Month, Week, and Day schedules showing tasks, issues, and milestones with drag-and-drop rescheduling and unscheduled task sidebar.
57. **Resource Utilization / Workload Heatmap View:** Grid showing every team member's daily/weekly allocated hours vs. available capacity (highlighting overloaded users in red).
58. **Portfolio / Program Roadmap View:** High-level timeline across multiple projects and project groups.
59. **My Work / Personal Agenda View:** Cross-project unified inbox of all tasks and issues assigned to the logged-in user, grouped by Today, Overdue, Upcoming, and Unscheduled.
60. **Matrix / Eisenhower View:** 2x2 prioritization grid (Urgent vs. Important / Impact vs. Effort).
61. **Gallery / Asset View:** Visual grid of all files, mockups, and attachments across tasks.
62. **Split-Screen View:** Viewing the Task List Sheet alongside the Gantt Chart or Detail Drawer simultaneously.
63. **Public View Sharing:** Generating read-only, optionally password-protected URLs for external stakeholders to view a Gantt chart or Kanban board.

Module 6: Custom Fields, Custom Views & Query Engine (10 Features)

64. **Custom Field Engine:** Adding project- or portal-level custom fields to Projects, Tasks, and Issues (Text, Multiline, Number, Currency, Decimal, Date, Date-Time, Picklist, Multi-Picklist, User Picklist, Checkbox, URL, Email, Phone, and Formula fields).
65. **Formula & Rollup Columns:** Calculating custom metrics inside the Task List Sheet (e.g., `Estimated Hours * Rate` or `Days Remaining`).
66. **Custom Status Workflows:** Creating custom statuses mapped to meta-states (`Open`, `In Progress`, `On Hold`, `Closed`) with custom hex colors.

67. **AST Filter Builder (Boolean Pattern Engine):** Filtering up to 10+ criteria combined with arbitrary logical patterns like `(1 AND 2) OR 3`.
68. **Relative Date Macros:** Dynamic filters for `today`, `yesterday`, `tomorrow`, `this_week`, `last_week`, `next_7_days`, `this_month`, `overdue`, and `unscheduled`.
69. **Hierarchy User Filters:** Filtering tasks by `current_login_user`, `direct_reports_of`, or `subordinates_of`.
70. **Saved Custom Views:** Saving filter + sort + column visibility + grouping configurations as Personal, Project, or Portal-wide Custom Views.
71. **Multi-Column Sorting:** Sorting by multiple fields simultaneously (e.g., `ASC(start_date), DESC(priority_sequence)`).
72. **Global Tag Management:** Color-coded tags shared across tasks, task lists, phases, and issues.
73. **Full-Text & Semantic Search:** Instant search across task titles, descriptions, comments, issue IDs, and attachment filenames.

Module 7: Issue / Bug Tracking & Quality Assurance (11 Features)

74. **Dedicated Issue Module:** Separate tracker for bugs, defects, and change requests alongside tasks.
75. **Bug-Specific Attributes:** `Severity` (Blocker, Critical, Major, Minor), `Reproducibility` (Always, Sometimes, Rarely, Unable), `Classification` (Security, Crash, Data Loss, UI/Usability, Performance), and `Module`.
76. **Milestone Linkage:** Tracking both `affected_milestone` (where the bug was found) and `release_milestone` (where the fix will ship).
77. **Issue-to-Task & Task-to-Issue Conversion:** Promoting a reported bug into a scheduled task or linking tasks that resolve an issue.
78. **SLA (Service Level Agreement) Engine:** Defining response-time and resolution-time targets based on issue severity, with multi-level escalation alerts when breached.
79. **Web-to-Issue Intake Forms:** Public or internal customizable forms where QA or customers submit bugs directly into a project.
80. **Email-to-Issue / Email-to-Task Alias:** Unique project email addresses that parse incoming emails and attachments into new issues or tasks.
81. **Git Commit & PR Linking:** Auto-transitioning issues when a GitHub/GitLab commit or PR references `Fixes #ISS-42`.
82. **Release Notes Generator:** Compiling all closed issues and tasks tied to a `release_milestone` into formatted release notes.
83. **Custom Issue Fields & Layouts:** Tailoring bug forms per engineering team.
84. **Duplicate Issue Merging:** Merging duplicate bug reports while preserving all reporters/followers.

Module 8: Time Tracking, Timesheets & Financial/EVM Budgeting (14 Features)

85. **Live Global Timer:** One-click start/pause/stop stopwatch in the top header that logs time directly to a task or issue.
86. **Manual Time Log Entry:** Logging hours (`HH:MM` or decimal) with date, billable/non-billable status, and work notes.
87. **Weekly / Monthly Timesheet Grid:** Spreadsheet-style timesheet view for rapid weekly hour entry across assigned tasks.
88. **Timesheet Approval Workflow:** Submitting weekly timesheets or individual time logs to project managers or client users for approval/rejection with comments.
89. **Time Log Restrictions:** Blocking future-date logging, capping daily max hours (e.g., 12h/day), or locking logs after a billing period closes.
90. **Project Budget Modes (`budget_type`):** Configuring budgets based on Project total, Milestone/Phase, Task, or User hours/cost.
91. **Billing Methods:** `project_hours`, `staff_hours`, `task_or_issue_hours`, or `fixed_cost`.
92. **Rate Cards (`rate_per_hour` & `cost_per_hour`):** Tracking both internal employee cost rate and external client billable rate in multiple currencies.
93. **Task & Phase Budget Allocations:** Setting `planned_cost`, `actual_cost`, `planned_hours`, `actual_hours`, `revenue_budget`, and budget alert thresholds (`threshold` %).
94. **Earned Value Management (EVM) Analytics:** Calculating Planned Value (PV), Earned Value (EV), Actual Cost (AC), Cost Variance (CV), Schedule Variance (SV), Cost Performance Index (CPI), and Schedule Performance Index (SPI).
95. **Expense Tracking:** Logging non-labor project expenses (software licenses, travel, hardware) against project budgets.
96. **Invoice & Retainer Generation:** Generating client invoices PDF/CSV directly from approved billable time logs and expenses.
97. **Budget Threshold Alerts:** Automated notifications when a task, phase, or project crosses 80% or 100% of its planned budget.
98. **Forecasted Cost/Hours to Completion:** Real-time projection based on current burn rate and remaining uncompleted work.

Module 9: Workflow Automation & Blueprint Studio (8 Features)

99. **Visual Blueprint Builder (State Machine):** Drag-and-drop canvas to design mandatory status transition paths for Tasks and Issues.

100. **Transition Guards (Before Transition):** Restricting *who* can click a transition button (e.g., only QA Lead can click "Verify Fix") and defining criteria required before the transition appears.
101. **Interactive Transition Prompts (During Transition):** Popping up a modal requiring mandatory fields, notes, attachments, or checklists *at the exact moment* a user changes status.
102. **Automated Post-Actions (After Transition):** Triggering field updates, email/in-app alerts, webhook calls, custom scripts, or child task creation upon transition completion.
103. **Business Rules & Workflow Rules**: Event-driven triggers (`On Create`, `On Field Edit`, `On Date Arrival`, `On Overdue`) that evaluate conditions and execute automated mutations.
104. **Outgoing Webhooks:** Firing signed HTTP payloads to external endpoints on task/issue/phase events.
105. **Custom Functions / Scripting Sandbox:** Running lightweight server-side JavaScript/TypeScript snippets on workflow events.
106. **Auto-Archiving & Hygiene Jobs:** Background cron jobs to auto-close stale issues or archive completed tasks after N days.

Module 10: Real-Time Collaboration, Feeds & File Repository (12 Features)

107. **Activity Feed & Status Stream:** Portal- and project-wide social feed showing real-time updates, status posts, and @mentions.
108. **Threaded Comments with Rich Media:** Markdown/Rich-text comments on tasks, task lists, phases, and issues with inline images, code blocks, and file attachments.
109. **Work-Item Embeds & Mentions**: Typing `@user` to notify members or `#TASK-102` to render a live interactive preview card inside any comment or doc.
110. **Project File Repository & Folder Tree:** Dedicated hierarchical cloud drive per project for storing deliverables, specs, and assets.
111. **Document Versioning & Check-In/Check-Out:** Uploading new versions of a file (`v1`, `v2`, `v3`) with version notes, diff history, and rollback.
112. **Visual Proofing & Annotations:** Pinning numbered feedback comments directly onto regions of images, PDFs, and video assets.
113. **Real-Time Collaborative Pages / Wiki:** Built-in Notion-style block editor (powered by Yjs/ProseMirror) for project documentation, meeting notes, and SOPs with live multi-cursor presence.
114. **Discussion Forums:** Long-form categorized discussion threads for architectural decisions and announcements.
115. **Built-In Project Chat Rooms**: Real-time direct and group messaging channels scoped to projects or teams.
116. **Notification Center (In-App, Email, Push)**: Granular notification preferences with digest batching and "Mark all as read" inbox.
117. **Live Presence Indicators:** Showing avatars of teammates currently viewing or editing the same Task List Sheet or Task Drawer.
118. **Document Approval & E-Signature Hooks:** Requesting formal sign-off on project spec documents.

Module 11: Analytics, BI Dashboards & Reporting (12 Features)

119. **Customizable Project Dashboard:** Drag-and-drop widget grid per project and at the portfolio level.
120. **Burndown & Burnup Charts:** Tracking remaining work (tasks, hours, or story points) against ideal velocity lines per Phase or Cycle.
121. **Planned vs. Actual Hours Report:** Comparing estimated work allocation against logged timesheet hours across tasks and users.
122. **Resource Utilization & Capacity Charts**: Visualizing billable vs. non-billable utilization percentages per team member.
123. **Task Status & Priority Distribution Charts:** Pie, doughnut, and stacked bar charts of open vs. closed work.
124. **Milestone / Phase Slippage Report:** Tracking historical deadline shifts across phases.
125. **Issue Aging & SLA Compliance Report:** Mean Time to Detect (MTTD) and Mean Time to Resolve (MTTR) analytics by severity.
126. **EVM Financial Dashboard:** Plotting PV, EV, and AC curves over the project timeline.
127. **Timesheet Summary & Billing Reports:** Pivot tables grouped by Client, Project, Phase, User, or Date range.
128. **Cross-Project Portfolio Rollup Dashboard:** Executive summary comparing budget, health, and completion % across all active projects.
129. **Custom Report Builder:** Ad-hoc chart and pivot table creator with scheduled PDF/CSV email delivery.
130. **Activity & Velocity Heatmaps:** GitHub-style contribution and task-completion heatmaps per team.

Module 12: Integrations, Importers, AI & Developer Platform (12 Features)

131. **Project Import / Migration Wizard:** Importing projects from Zoho Projects JSON/CSV, Jira, Asana, Trello, ClickUp, and Microsoft Project (`.mpp` / `.xml`).
132. **Comprehensive Data Export:** Exporting Task List Sheets, Gantt charts, and Timesheets to CSV, Excel (`.xlsx`), PDF, and JSON.

133. **Git Integrations:** Bi-directional sync with GitHub, GitLab, and Bitbucket (branches, commits, pull requests, CI status).
134. **Chat & Calendar Sync:** Slack, Microsoft Teams, Google Calendar, and Outlook two-way calendar sync (iCal feeds).
135. **Cloud Storage Connectors:** Attaching files from Google Drive, OneDrive, Dropbox, and Box.
136. **Accounting & CRM Connectors:** Syncing billable hours to QuickBooks, Xero, Stripe, or HubSpot/CRM deals.
137. **AI Project Co-Pilot:** Generating full WBS (Phases -> Task Lists -> Tasks -> Subtasks) from a natural language product brief.
138. **AI Risk & Bottleneck Predictor:** Flagging tasks likely to miss deadlines based on dependency chains and assignee overload.
139. **AI Thread & Meeting Summarizer:** Condensing 50+ task comments or wiki pages into action items.
140. **REST & GraphQL Public API:** Full-featured, rate-limited API with Personal Access Tokens (PATs) and OAuth2 apps.
141. **Interactive API Documentation (OpenAPI / Swagger):** Built-in developer portal.
142. **Desktop & Mobile PWA Wrapper:** Offline-capable service worker and responsive mobile layouts.

3. Complete Screen & UI Surface Inventory (54 Screens & Views)

To house these 142 features cleanly without overwhelming users, the UI is divided into **7 logical screen groups** totaling **54 distinct screens, views, drawers, and complex modals**:

Group A: Authentication & Onboarding (6 Screens)

1. **Login Screen** (Email/Password, SSO, Magic Link)
2. **Registration & Account Verification Screen**
3. **Forgot / Reset Password Screen**
4. **2FA Verification & Setup Screen**
5. **Portal Creation & Onboarding Wizard** (Company Name, URL Slug, Timezone, Business Hours, Initial Project Template)
6. **Portal Switcher / Organization Selector Screen**

Group B: Global Portal Workspace (8 Screens)

7. **Global Home / Executive Command Center** (Personal widgets, overdue alerts, recent projects, quick timers)
8. **My Work / Personal Agenda Screen** (Unified cross-project Tasks, Issues, Reminders, and Calendar for the current user)
9. **Global Feed & Activity Stream Screen** (Company-wide updates, @mentions, status posts)
10. **Project Portfolio Directory Screen** (List, Card, and Rollup Matrix of all Active, Archived, and Template projects)
11. **Project Groups / Portfolio Roadmap Screen** (Cross-project Gantt timeline & budget health)
12. **Global Timesheets & Approvals Hub** (Cross-project weekly timesheet grid and manager approval queue)
13. **Global Resource Utilization Heatmap Screen** (Company-wide capacity vs. workload planner)
14. **Global Reports & BI Analytics Hub** (50+ pre-built & custom cross-project reports)

Group C: Project-Scoped Core Views (12 Screens)

15. **Project Dashboard Overview Screen** (Health score, burn rate, EVM metrics, phase progress, recent activity)
16. **Task List Sheet View (Primary WBS Grid)** (Collapsible Phase -> Task List -> Task -> Subtask spreadsheet with inline cell editors and summary rows)
17. **Task Kanban Board View** (Drag-and-drop columns with swimlanes by Phase, Task List, Assignee, or Priority)
18. **Interactive Gantt Chart & Critical Path View** (Split pane: WBS tree on left, interactive timeline canvas with dependency lines & baselines on right)
19. **Project Calendar View** (Month/Week/Day calendar of tasks, phases, and issues)
20. **Phases / Milestones Directory View** (Dedicated board/list of all project phases, budgets, and sequential chains)
21. **Project Issues / Bug Tracker View** (Sheet, Kanban, and SLA escalation views specifically for project bugs)
22. **Project Timesheet & Log Hours View** (List and calendar of all billable/non-billable hours logged in the project)
23. **Project Finance, Budget & Expenses View** (Cost vs. Revenue vs. EVM dashboard, rate cards, expenses, and invoice generator)
24. **Project File Repository & Asset Manager View** (Folder tree, file grid/list, version history, and visual proofing viewer)

25. **Project Pages / Wiki Knowledge Base View** (Nested document tree + real-time collaborative block editor)
26. **Project Forums & Chat Rooms View** (Threaded discussions and real-time project chat channels)

Group D: Slide-Over Detail Drawers & Inspection Panels (8 Surfaces)

(Crucial UX pattern: clicking a task or issue opens a 60% width right-hand slide-over drawer so the user never loses their scroll position in the Task List Sheet or Kanban board.) 27. **Task Detail Slide-Over Drawer** (Tabs for Subtasks, Checklists, Dependencies, Work Allocation, Time Logs, Comments, Attachments, Blueprint Transitions, and Activity History) 28. **Issue / Bug Detail Slide-Over Drawer** (Severity, Reproducibility, Affected/Release Milestone, SLA timer, Linked Tasks, Git Commits, and Comments) 29. **Phase / Milestone Detail Drawer** (Associated Task Lists, Phase Budget/EVM, Slippage History) 30. **Task List Detail & Settings Drawer** (Phase assignment, internal/external flag, sequence, rollover tool) 31. **File Preview & Visual Proofing Lightbox** (Full-screen PDF/Image/Video annotator with version switcher) 32. **User Workload & Availability Mini-Drawer** (Inspecting a teammate's daily schedule and leaves before assigning a task) 33. **Notification Inbox Slide-Over Drawer** (Filtered by Unread, @Mentions, Assigned to Me, Approvals) 34. **AI Project Assistant Side-Panel** (Natural language WBS generator, risk analyzer, and thread summarizer)

Group E: High-Complexity Interactive Modals (10 Surfaces)

35. **Create / Edit Project Modal** (General info, Owner, Layout, Budget Config, Copy-From Template selector)
36. **Quick-Create Task / Issue Modal** (Triggered anywhere via `C` shortcut)
37. **Work Allocation Matrix Modal** (Configuring Standard vs. Flexible daily hours/percentages across multiple task owners)
38. **Recurrence & Reminder Configuration Modal** (Cron builder and notification recipient selector)
39. **Advanced AST Filter Builder & Save Custom View Modal** (Visual `(1 AND 2) OR 3` criteria builder)
40. **Blueprint Transition Execution Modal** (Dynamic form rendered during a state transition requiring specific inputs/attachments)
41. **Time Log & Timer Modal** (Logging hours, selecting billing status, attaching work notes)
42. **Bulk Edit / Move / Rollover Modal** (Applying batch mutations to 50+ selected tasks)
43. **Import / Export Wizard Modal** (Column mapping for CSV/Jira/Zoho/MPP imports)
44. **Global Command Palette (`Cmd+K`)** (Instant fuzzy search and action runner)

Group F: Automation & Customization Studio (4 Screens)

45. **Visual Blueprint Editor Screen** (Node-and-edge canvas for designing state transitions, guards, and post-actions)
46. **Layout & Custom Fields Builder Screen** (Drag-and-drop field designer for Projects, Tasks, and Issues)
47. **Workflow & Business Rules Builder Screen** (Event -> Condition -> Action rule creator and SLA escalation builder)
48. **Webhooks & Custom Functions Console** (Endpoint logs, retry queues, and script editor)

Group G: Portal & Project Administration (6 Screens)

49. **Users, Client Companies & Teams Directory Screen** (Invitations, deactivation, team grouping)
50. **Roles, Profiles & Permission Matrix Screen** (Granular CRUD + field-level security grid)
51. **Portal Settings & Business Calendar Screen** (Working hours, holidays, currencies, date/time formats, custom domains)
52. **Integrations & API Keys Marketplace Screen** (GitHub, Slack, Google Drive, OAuth apps, Personal Access Tokens)
53. **Audit Log & Recycle Bin (Trash) Screen** (Filterable security trail and deleted item recovery)
54. **Subscription, Quotas & Storage Billing Screen** (Seat count, file storage usage, plan tiers)

4. How Many Files Are Required to Build This Project?

In a modern TypeScript full-stack monorepo (using **Next.js App Router** for the frontend, **NestJS** for the modular API backend, a **Node.js/Hocuspocus/Socket.io** real-time collaboration service, **BullMQ** workers, and **Prisma/Drizzle** with PostgreSQL), the file count scales predictably based on your target milestone:

Build Stage	Scope Included	Estimated Source Files	Estimated Lines of Code (LOC)
Stage 1: Core WBS MVP	Auth, Portals, Projects, Phases, Task Lists, Tasks/Subtasks, **Task List Sheet View**, Kanban View, Comments, Basic File Uploads	**~210 – 260 files**	~18,000 – 25,000 LOC

Build Stage	Scope Included	Estimated Source Files	Estimated Lines of Code (LOC)
Stage 2: Zoho Projects Parity	Everything in Stage 1 + **Interactive Gantt Chart**, Dependencies, **Issue Tracker & SLAs**, **Timesheets & Timers**, Custom Fields, AST Filter Engine & Saved Views, Project Templates, Project Docs/Pages	**~520 – 650 files**	~48,000 – 65,000 LOC
Stage 3: Enterprise Suite	Everything in Stage 2 + **Visual Blueprint State Machine**, **EVM & Financial Budgeting**, Resource Heatmap, Real-Time Collaborative Wiki (Yjs), Webhooks/Business Rules, BI Report Builder, Importers, AI Co-Pilot	**~920 – 1,150 files**	~85,000 – 120,000+ LOC

Breakdown of the ~600 Files (Stage 2 Target) by Layer

To see exactly where those files live in a production repository:

Workspace / Package	File Count (Stage 2)	File Count (Stage 3)	What These Files Are
`apps/web` **(Next.js Frontend)**	~240 files	~420 files	54 Route/Page files, ~110 feature components (Sheet Grid, Gantt Canvas, Kanban, Drawers), ~35 Zustand/TanStack Query hooks, ~40 modals & forms.
`apps/api` **(NestJS Core Backend)**	~140 files	~260 files	~18 domain modules (`projects`, `phases`, `tasklists`, `tasks`, `issues`, `timesheets`, `blueprints`, etc.) × 6–8 files each (`controller`, `service`, `repository`, `dto`, `guard`, `spec`).
`apps/realtime` **(WebSocket / Yjs Server)**	~20 files	~35 files	Presence rooms, live Task List Sheet cell locking, Yjs collaborative document sync, live notification broadcaster.
`apps/worker` **(Background Jobs)**	~25 files	~50 files	Recurring task generator, reminder scheduler, SLA escalation checker, Gantt dependency cascade calculator, webhook dispatcher, CSV/MPP importer.
`packages/database` **(ORM & Migrations)**	~30 files	~50 files	Modular database schemas (35–45 tables), SQL migrations, PostgreSQL sequence/advisory lock helpers, seed scripts.
`packages/shared-types` & `validators`	~35 files	~65 files	Shared Zod schemas, TypeScript DTO interfaces, enum constants, RBAC permission matrices.
`packages/query-engine` **(AST Filter & WBS)**	~15 files	~25 files	Parser that compiles `(1 AND 2) OR 3` filter JSON + relative date macros (`next_7_days`) into safe parameterized SQL queries, plus critical-path graph solvers.
`packages/ui` **(Design System)**	~45 files	~70 files	Primitives (Buttons, Virtualized DataGrid, DateRangePicker, UserAvatarGroup, RichTextEditor/TipTap extensions, SlideOverDrawer, CommandPalette).
Root Config, Docker & CI/CD	~20 files	~35 files	`turbo.json`, `pnpm-workspace.yaml`, `docker-compose.yml`, GitHub Actions workflows, Nginx/Caddy configs.
Total Repository Files	**~570 files**	**~1,010 files**	**Full production-grade Zoho Projects alternative**

5. Complete File Repository Structure (Monorepo Blueprint)

Below is the complete, production-ready **Turborepo + pnpm** file repository structure. It cleanly separates applications (`apps/`) from shared domain libraries (`packages/`) so that your Frontend, Backend API, Real-Time Server, and Background Workers share 100% of their TypeScript types, validation schemas, and database models.

```
zoho-projects-alternative/
├── .github/
│   └── workflows/
│       ├── ci-lint-typecheck.yml          # Runs ESLint, Prettier, and tsc across all packages
│       ├── ci-test-suite.yml              # Unit + integration tests against ephemeral Postgres/Redis
│       └── cd-release-docker.yml          # Builds and pushes Docker images for web, api, realtime, worker
├── docker/
│   ├── api.Dockerfile                     # Multi-stage production build for NestJS API
│   ├── web.Dockerfile                     # Multi-stage Next.js standalone build
│   ├── realtime.Dockerfile                # WebSocket & Hocuspocus Yjs collaboration server build
│   ├── worker.Dockerfile                  # BullMQ background processor build
│   └── nginx/
│       └── default.conf                   # Reverse proxy routing /api, /ws, and /* to respective containers
├── docker-compose.yml                     # Local dev stack: Postgres 16, Redis 7, MinIO (S3), Mailpit
├── docker-compose.prod.yml                # Self-hostable production orchestration
├── package.json                           # Root workspace scripts (dev, build, db:migrate, lint)
├── pnpm-workspace.yaml                    # Declares apps/* and packages/*
├── turbo.json                             # Turborepo pipeline caching & task dependency graph
```

```
├── tsconfig.base.json                    # Strict shared TypeScript compiler configuration
├── .env.example                          # Template environment variables
│
├── apps/
│   ├── web/                              # [FRONTEND] Next.js 15+ (App Router) + React + Tailwind + TanStack
│   │   ├── package.json
│   │   ├── next.config.ts
│   │   ├── tailwind.config.ts
│   │   ├── public/
│   │   │   ├── icons/
│   │   │   └── sounds/                   # Subtle timer start/stop & notification chimes
│   │   └── src/
│   │       ├── app/                      # Next.js App Router (Screens & Layouts)
│   │       │   ├── layout.tsx            # Root providers (QueryClient, Theme, Hotkeys, Toast)
│   │       │   ├── (auth)/
│   │       │   │   ├── login/page.tsx
│   │       │   │   ├── register/page.tsx
│   │       │   │   ├── forgot-password/page.tsx
│   │       │   │   ├── verify-2fa/page.tsx
│   │       │   │   └── onboarding/page.tsx
│   │       │   └── (portal)/
│   │       │       └── [portalSlug]/     # Multi-tenant Portal Scope
│   │       │           ├── layout.tsx    # Sidebar, Global Header, Active Timer Bar, Command Palette
│   │       │           ├── home/page.tsx                      # Screen 7: Executive Command Center
│   │       │           ├── my-work/page.tsx                   # Screen 8: Unified Personal Agenda
│   │       │           ├── feed/page.tsx                      # Screen 9: Global Activity Stream
│   │       │           ├── projects/
│   │       │           │   ├── page.tsx                      # Screen 10: Project Portfolio Directory
│   │       │           │   ├── groups/page.tsx               # Screen 11: Portfolio Roadmap & Rollups
│   │       │           │   └── [projectId]/                  # Project Workspace Scope
│   │       │           │       ├── layout.tsx               # Project Header, Module Tabs, View Switcher
│   │       │           │       ├── dashboard/page.tsx        # Screen 15: Project Health & EVM Overview
│   │       │           │       ├── tasks/
│   │       │           │       │   ├── page.tsx             # Redirects to active/default view (Sheet/Kanban)
│   │       │           │       │   ├── sheet/page.tsx       # Screen 16: Task List Sheet View (WBS Grid)
│   │       │           │       │   ├── kanban/page.tsx      # Screen 17: Task Kanban Board View
│   │       │           │       │   ├── gantt/page.tsx       # Screen 18: Interactive Gantt & Critical Path
│   │       │           │       │   └── calendar/page.tsx    # Screen 19: Project Calendar View
│   │       │           │       ├── phases/page.tsx           # Screen 20: Phases / Milestones Directory
│   │       │           │       ├── issues/
│   │       │           │       │   ├── page.tsx             # Screen 21: Issue / Bug Tracker Sheet & Board
│   │       │           │       │   └── slas/page.tsx        # SLA Escalation Monitor
│   │       │           │       ├── timesheets/page.tsx      # Screen 22: Project Timesheets & Approval Queue
│   │       │           │       ├── finance/page.tsx         # Screen 23: Project Budget, EVM, Expenses & Invoices
│   │       │           │       ├── files/page.tsx           # Screen 24: File Repository, Folders & Proofing
│   │       │           │       ├── pages/
│   │       │           │       │   ├── page.tsx             # Screen 25: Project Wiki Tree
│   │       │           │       │   └── [pageId]/page.tsx    # Collaborative Yjs Document Editor
│   │       │           │       ├── discussions/page.tsx     # Screen 26: Forums & Project Chat Rooms
│   │       │           │       └── settings/
│   │       │           │           ├── general/page.tsx
│   │       │           │           ├── members/page.tsx
│   │       │           │           ├── custom-fields/page.tsx
│   │       │           │           └── blueprints/page.tsx
│   │       │           ├── timesheets/page.tsx               # Screen 12: Global Cross-Project Timesheets
│   │       │           ├── workload/page.tsx                 # Screen 13: Global Resource Utilization Heatmap
│   │       │           ├── reports/
│   │       │           │   ├── page.tsx                      # Screen 14: Global BI Analytics Hub
│   │       │           │   └── builder/page.tsx              # Custom Report & Pivot Table Builder
│   │       │           ├── automation/
│   │       │           │   ├── blueprints/[id]/page.tsx      # Screen 45: Visual Blueprint State-Machine Canvas
│   │       │           │   ├── layouts/page.tsx              # Screen 46: Layout & Custom Field Designer
│   │       │           │   ├── rules/page.tsx                # Screen 47: Business Rules & SLA Engine
│   │       │           │   └── webhooks/page.tsx             # Screen 48: Webhooks & Custom Functions
│   │       │           └── admin/
│   │       │               ├── users/page.tsx                # Screen 49: Users, Clients & Teams
│   │       │               ├── roles/page.tsx                # Screen 50: Roles & Profile Permission Matrix
│   │       │               ├── portal-settings/page.tsx      # Screen 51: Business Hours, Holidays, Branding
│   │       │               ├── integrations/page.tsx         # Screen 52: GitHub, Slack, Imports, API Keys
│   │       │               ├── audit-logs/page.tsx           # Screen 53: Audit Trail & Trash Recovery
│   │       │               └── billing/page.tsx              # Screen 54: Subscription & Storage Quotas
│   │       │
│   │       ├── features/                 # Feature-Sliced Domain Modules (UI + State + Hooks)
│   │       │   ├── task-sheet/           # The Zoho-style Task List Sheet Engine
```

```
│   │   │   │   ├── components/
│   │   │   │   │   ├── TaskSheetGrid.tsx                  # Virtualized WBS Tree-Table (Phase -> List -> Task -> Su
│   │   │   │   │   ├── PhaseGroupHeaderRow.tsx            # Collapsible Phase banner with progress & budget summary
│   │   │   │   │   ├── TaskListSectionRow.tsx             # Task List header with internal/external badge & rollove
│   │   │   │   │   ├── TaskSheetRow.tsx                   # Individual task/subtask row with indentation guides
│   │   │   │   │   ├── InlineQuickAddRow.tsx              # Keyboard-driven bottom row for rapid task entry
│   │   │   │   │   ├── SheetBulkActionBar.tsx             # Floating bar when multiple checkboxes are selected
│   │   │   │   │   └── cells/                             # Specialized Inline Cell Editors
│   │   │   │   │       ├── StatusCellEditor.tsx
│   │   │   │   │       ├── PriorityCellEditor.tsx
│   │   │   │   │       ├── MultiOwnerWorkCellEditor.tsx   # Assigns users + Standard/Flexible work hours
│   │   │   │   │       ├── DateDurationCellEditor.tsx     # Start/End date picker synced with duration
│   │   │   │   │       ├── CompletionSliderCell.tsx       # 0-100% progress bar cell
│   │   │   │   │       ├── DependencyCellEditor.tsx       # Predecessor/Successor code editor (e.g., 14FS+2d)
│   │   │   │   │       └── CustomFieldDynamicCell.tsx     # Renders picklist, formula, currency, checkbox
│   │   │   │   ├── hooks/
│   │   │   │   │   ├── useSheetKeyboardNav.ts             # Excel-style arrow key, Enter, Tab, Copy/Paste navigatio
│   │   │   │   │   ├── useWbsTreeFlattener.ts             # Flattens hierarchical WBS for 60fps virtual scrolling
│   │   │   │   │   └── useOptimisticTaskMutation.ts       # Zero-latency UI updates with rollback on error
│   │   │   │   └── store/
│   │   │   │       └── useSheetViewStore.ts               # Column widths, frozen columns, expanded/collapsed node
│   │   │   │
│   │   │   ├── gantt-chart/           # Interactive Timeline & Critical Path Engine
│   │   │   │   ├── components/
│   │   │   │   │   ├── GanttSplitViewport.tsx
│   │   │   │   │   ├── GanttTimelineCanvas.tsx            # SVG/Canvas rendering of bars, milestones, and weekends
│   │   │   │   │   ├── DependencySVGOverlay.tsx           # Bezier/Orthogonal arrows for FS, SS, FF, SF links
│   │   │   │   │   ├── CriticalPathToggle.tsx             # Highlights zero-slack chain in red
│   │   │   │   │   └── BaselineGhostBars.tsx              # Renders original baseline schedule underneath live bars
│   │   │   │   └── hooks/
│   │   │   │       └── useGanttDragResize.ts              # Drag-to-move, drag-edge-to-resize, drag-pin-to-link
│   │   │   │
│   │   │   ├── kanban-board/          # Drag-and-Drop Board & Swimlane Engine
│   │   │   │   ├── components/
│   │   │   │   │   ├── KanbanBoardCanvas.tsx
│   │   │   │   │   ├── KanbanSwimlaneGroup.tsx
│   │   │   │   │   ├── KanbanColumn.tsx
│   │   │   │   │   └── KanbanTaskCard.tsx
│   │   │   │   └── hooks/
│   │   │   │       └── useKanbanDragDrop.ts               # Fractional indexing (LexoRank) for sequence updates
│   │   │   │
│   │   │   ├── task-drawer/           # Slide-Over Inspection Drawer
│   │   │   │   ├── TaskDetailDrawer.tsx
│   │   │   │   ├── SubtasksTabPanel.tsx
│   │   │   │   ├── WorkAllocationModal.tsx
│   │   │   │   ├── RecurrenceReminderPanel.tsx
│   │   │   │   ├── BlueprintTransitionBar.tsx             # Renders valid state-machine buttons for the task
│   │   │   │   └── TaskActivityDiffStream.tsx             # Audit trail of field changes + threaded comments
│   │   │   │
│   │   │   ├── issues-tracker/        # Bug Tracking & SLA Components
│   │   │   ├── timesheets-finance/    # Global Timer, Weekly Grid, EVM Charts, Invoice Builder
│   │   │   ├── blueprint-studio/      # ReactFlow Visual State-Machine Editor
│   │   │   ├── filter-custom-views/   # Visual AST Filter Builder ((1 AND 2) OR 3)
│   │   │   ├── file-vault/            # Folder Tree, Version Uploader, Image/PDF Proofing Canvas
│   │   │   └── wiki-pages/            # TipTap + Yjs Real-Time Block Editor
│   │   │
│   │   ├── lib/
│   │   │   ├── api-client.ts          # Typed fetch/ky wrapper with auth token refresh & portal header
│   │   │   ├── socket-client.ts       # WebSocket connection manager for live updates
│   │   │   └── indexeddb-cache.ts     # Local-first cache for instant view switching
│   │   └── stores/
│   │       ├── useGlobalTimerStore.ts # Persists running stopwatch across page navigations
│   │       └── useDrawerStore.ts      # Controls slide-over drawers via URL query params (?taskId=123)
│   │
│   ├── api/                           # [BACKEND API] NestJS Modular Architecture
│   │   ├── package.json
│   │   ├── nest-cli.json
│   │   ├── tsconfig.json
│   │   └── src/
│   │       ├── main.ts                # Bootstrap, Swagger/OpenAPI setup, CORS, Global ValidationPipe
│   │       ├── app.module.ts          # Root DI container importing all 18 domain modules
│   │       ├── common/
│   │       │   ├── decorators/
│   │       │   │   ├── current-user.decorator.ts
```

```
│   │   │   │   ├── portal-scope.decorator.ts
│   │   │   │   └── require-permissions.decorator.ts
│   │   │   ├── guards/
│   │   │   │   ├── jwt-auth.guard.ts
│   │   │   │   ├── portal-membership.guard.ts
│   │   │   │   ├── rbac-profile.guard.ts
│   │   │   │   └── client-visibility.guard.ts            # Enforces internal vs. external flag scoping
│   │   │   ├── interceptors/
│   │   │   │   ├── audit-log.interceptor.ts              # Automatically diffs & records field mutations
│   │   │   │   └── field-masking.interceptor.ts          # Strips cost_per_hour if user lacks financial permission
│   │   │   └── filters/
│   │   │       └── global-exception.filter.ts
│   │   │
│   │   └── modules/                  # 18 Core Backend Domain Modules
│   │       ├── auth/                 # Login, OAuth, SAML, 2FA, Sessions
│   │       ├── portals/              # Portal CRUD, Business Hours, Holidays, Custom Domains
│   │       ├── users-teams/          # Portal Members, Client Companies, Teams, Leave/Availability
│   │       ├── roles-permissions/    # Roles, Profiles, Field-Level Security Policies
│   │       ├── projects/             # Projects, Project Groups, Templates, Roll-up Aggregation, Clone
│   │       │   ├── projects.module.ts
│   │       │   ├── projects.controller.ts
│   │       │   ├── projects.service.ts
│   │       │   ├── project-cloner.service.ts             # Deep-clones phases, lists, tasks & relative dates
│   │       │   └── dto/
│   │       ├── phases/               # Phases (Milestones), Sequential Chaining, Phase Budgets
│   │       ├── task-lists/           # Task Lists, Default Lists, Sequence Reordering, Rollover
│   │       │   ├── task-lists.module.ts
│   │       │   ├── task-lists.controller.ts
│   │       │   ├── task-lists.service.ts
│   │       │   └── dto/
│   │       ├── tasks/                # Tasks, Subtasks, Owners/Work Allocation, Dependencies, Recurrence
│   │       │   ├── tasks.module.ts
│   │       │   ├── tasks.controller.ts
│   │       │   ├── tasks.service.ts
│   │       │   ├── task-dependency.service.ts            # Cycle detection (DAG) & date auto-cascade
│   │       │   ├── task-work-allocation.service.ts       # Standard vs. Flexible work calculator
│   │       │   ├── task-sequence.service.ts              # LexoRank / sequence ordering + PG advisory locks
│   │       │   └── dto/
│   │       ├── issues/               # Bug Tracker, Severity, Reproducibility, SLA Timers, Git hooks
│   │       ├── timesheets/           # Time Logs, Active Timers, Weekly Sheets, Approval Workflow
│   │       ├── budgets-evm/          # Cost/Revenue/Hourly Budgets, Rate Cards, EVM (PV/EV/AC/CPI/SPI), Invoices
│   │       ├── custom-fields-layouts/ # Dynamic Custom Fields, Picklist Options, Formula Evaluator
│   │       ├── custom-views-filters/  # Saved Views, AST Filter Compiler ((1 AND 2) OR 3)
│   │       ├── blueprints-automation/ # State Machine Engine, Transition Guards, Business Rules, Webhooks
│   │       ├── files-proofing/       # S3 Multipart Uploads, Folder Tree, Versioning, Proofing Pins
│   │       ├── collaboration/        # Comments, @Mentions, Activity Feeds, Forums, Wiki Pages, Notifications
│   │       ├── reports-analytics/    # Burndown, Resource Utilization, Planned vs. Actual, Custom Pivots
│   │       └── integrations-io/      # CSV/Zoho/Jira/MPP Importers, GitHub/Slack Webhooks, Public API
│   │
│   ├── realtime/                     # [REAL-TIME SERVER] WebSocket + Yjs / Hocuspocus
│   │   ├── package.json
│   │   └── src/
│   │       ├── server.ts             # Bootstraps Socket.io + Hocuspocus Yjs server with Redis adapter
│   │       ├── auth-middleware.ts    # Validates JWT & portal/project access on handshake
│   │       ├── rooms/
│   │       │   ├── project-sheet.room.ts  # Broadcasts live cell edits, row locks & cursor presence in Sheet View
│   │       │   ├── project-chat.room.ts   # Real-time project chat messages & typing indicators
│   │       │   └── user-notify.room.ts    # Instant toast & badge counter push
│   │       └── yjs/
│   │           └── wiki-persistence.ts    # Triple-writes Yjs binary, ProseMirror JSON, and HTML to Postgres
│   │
│   └── worker/                       # [BACKGROUND WORKER] BullMQ + Redis Cron & Queue Processors
│       ├── package.json
│       └── src/
│           ├── main.ts
│           └── processors/
│               ├── recurrence.processor.ts        # Spawns next task occurrence daily or on task close
│               ├── reminder.processor.ts          # Evaluates before/after/on reminder rules & sends emails/push
│               ├── sla-escalation.processor.ts    # Monitors open issue SLAs & triggers breach escalations
│               ├── gantt-cascade.processor.ts     # Recalculates large dependency graphs & parent phase rollups
│               ├── evm-rollup.processor.ts        # Computes project/phase Earned Value & budget threshold alerts
│               ├── webhook-dispatch.processor.ts  # Delivers outgoing webhooks with exponential backoff retries
│               └── import-export.processor.ts     # Parses large MPP/CSV/Jira imports and generates PDF invoices
│
```

```
└── packages/                           # [SHARED MONOREPO LIBRARIES]
    ├── database/                       # Prisma / Drizzle ORM + PostgreSQL Schema & Migrations
    │   ├── package.json
    │   ├── prisma/
    │   │   ├── schema/                 # Multi-file Modular Database Schema (42 Core Tables)
    │   │   │   ├── 01_portal_auth.prisma          # Portals, Users, Sessions, BusinessHours, Holidays
    │   │   │   ├── 02_rbac_teams.prisma           # Roles, Profiles, Permissions, Teams, ClientCompanies
    │   │   │   ├── 03_projects.prisma             # Projects, ProjectGroups, ProjectMembers, ProjectBudgets
    │   │   │   ├── 04_wbs_phases_lists.prisma     # Phases (Milestones), TaskLists, TaskListFollowers
    │   │   │   ├── 05_tasks_subtasks.prisma       # Tasks, TaskOwnersWork, TaskDependencies, Checklists, Recurrences, R
    │   │   │   ├── 06_issues_slas.prisma          # Issues, IssueFollowers, SlaPolicies, SlaBreachLogs
    │   │   │   ├── 07_timesheets_finance.prisma   # TimeLogs, ActiveTimers, Expenses, RateCards, Invoices
    │   │   │   ├── 08_custom_fields_views.prisma  # Layouts, CustomFields, CustomFieldOptions, CustomViews, Tags
    │   │   │   ├── 09_blueprints_rules.prisma     # Blueprints, BlueprintStates, BlueprintTransitions, BusinessRules, W
    │   │   │   └── 10_collab_files_audit.prisma   # Comments, Attachments, Folders, FileVersions, ProofAnnotations, Wik
    │   │   ├── migrations/
    │   │   └── seed.ts                 # Seeds default roles, statuses, and sample project template
    │   └── src/
    │       ├── client.ts               # Singleton DB connection pool
    │       └── sequence-lock.ts        # PostgreSQL pg_advisory_xact_lock for atomic TASK-101 / ISS-42 numbering
    │
    ├── shared-types/                   # Isomorphic Zod Schemas & TypeScript Contracts
    │   ├── package.json
    │   └── src/
    │       ├── index.ts
    │       ├── enums.ts                # Priority, BillingType, WorkType, DependencyType, FlagType (internal/externa
    │       ├── task.schema.ts          # Shared create/update task payload validation
    │       ├── issue.schema.ts
    │       ├── filter-ast.schema.ts    # Validates {"criteria": [...], "pattern": "(1 AND 2) OR 3"}
    │       └── budget-evm.schema.ts
    │
    ├── query-engine/                   # Core Algorithmic & Domain Logic Library
    │   ├── package.json
    │   └── src/
    │       ├── ast-filter-compiler.ts  # Parses boolean pattern strings & date macros into SQL WHERE clauses
    │       ├── critical-path-solver.ts # Topological sort + forward/backward pass for Gantt Critical Path & Slack
    │       ├── business-day-math.ts    # Adds/subtracts durations skipping portal weekends & holidays
    │       ├── evm-calculator.ts       # Pure functions for PV, EV, AC, CPI, SPI, EAC, ETC
    │       └── formula-parser.ts       # Safe AST evaluator for custom formula columns in Task List Sheets
    │
    └── ui/                             # Shared Design System & Complex UI Primitives
        ├── package.json
        └── src/
            ├── components/
            │   ├── button.tsx
            │   ├── slide-over-drawer.tsx
            │   ├── virtual-tree-grid.tsx
            │   ├── rich-text-editor/   # TipTap with @mention, #task-embed, and slash-command extensions
            │   └── command-palette.tsx
            └── index.ts
```

6. How the Files Will Be Made: The 6-Layer Construction Methodology

To build a system of this scale without drowning in technical debt or circular dependencies, files should be created using a strict **Bottom-Up, Schema-First Pipeline** across 6 sequential layers:

Layer 1: Modular Database Schema Files (`packages/database/prisma/schema/*.prisma`)

Instead of a single massive 3,000-line schema file, split the database into **10 domain schema files** (~42 tables total).

- **How they are made:**
  1. Define the **Multi-Tenant Root** (`Portal`) and attach `portal_id` to every high-level entity for strict tenant isolation and index performance.
  2. Build the **WBS Chain**: `Project` -> `Phase` (`Milestone`) -> `TaskList` -> `Task` (with self-referencing `parent_task_id` and `root_task_id` for infinite subtasks).
  3. Add **JSONB + Indexed Hybrid Storage** for Custom Fields: store native fields (`status_id`, `priority`, `start_date`, `end_date`, `completion_percentage`, `flag`) as typed relational columns, and store dynamic custom field values in an indexed PostgreSQL `JSONB` column (`custom_fields_data`) backed by a GIN index so the AST Filter Engine can query custom fields in milliseconds.
  4. Write `sequence-lock.ts` using PostgreSQL `pg_advisory_xact_lock(project_id)` so concurrent users creating tasks at the exact same millisecond receive sequential keys (`PRJ-101`, `PRJ-102`) with zero race conditions.

Layer 2: Shared Contracts & Zod Schemas (`packages/shared-types/src/*.schema.ts`)

Before writing a single API route or React component, define the exact shapes of every entity in `packages/shared-types`.

- **How they are made:**
    1. Each domain gets a `.schema.ts` file exporting Zod validation objects (`CreateTaskSchema`, `UpdateTaskSchema`, `TaskFilterQuerySchema`).
    2. Export inferred TypeScript types (`export type CreateTaskInput = z.infer<typeof CreateTaskSchema>`).
    3. Both `apps/api` (for request body validation in NestJS controllers) and `apps/web` (for React Hook Form and inline Task List Sheet cell validation) import from this exact same file.

Layer 3: Pure Algorithmic Engines (`packages/query-engine/src/*.ts`)

A project management tool has four complex mathematical/logical problems that must live in pure, unit-tested TypeScript files independent of the web framework:

1. **`ast-filter-compiler.ts`:** Tokenizes Zoho-style filter patterns like `(1 AND 2) OR 3`, resolves relative date macros (`today`, `next_7_days`, `overdue`) against the portal's timezone, and outputs a parameterized Prisma/SQL query object.
2. **`business-day-math.ts`:** Given a task `start_date`, a `duration` (e.g., `5 days`), and the portal's `BusinessHours` + `Holidays` calendar, computes the exact `end_date` while skipping non-working days.
3. **`critical-path-solver.ts`:** Builds a Directed Acyclic Graph (DAG) of all tasks and `TaskDependencies` (`FS`, `SS`, `FF`, `SF`), detects circular dependency errors, runs a forward pass (Early Start / Early Finish) and backward pass (Late Start / Late Finish), identifies tasks with `Total Float == 0` (the Critical Path), and calculates the date cascade when a predecessor task slips.
4. **`evm-calculator.ts`:** Computes Planned Value, Earned Value (`completion_percentage * planned_budget`), Actual Cost (from `TimeLogs * cost_per_hour` + `Expenses`), and CPI/SPI ratios.

Layer 4: Backend API Modules (`apps/api/src/modules/*`)

Every backend module follows a standardized **5-file generator pattern**:

1. **`{module}.controller.ts`:** REST endpoints scoped identically to enterprise hierarchy (`/api/v1/portals/:portalId/projects/:projectId/tasks`), decorated with `@UseGuards(JwtAuthGuard, PortalMembershipGuard, RbacProfileGuard, ClientVisibilityGuard)`.
2. **`{module}.service.ts`:** Core business logic, calling `packages/query-engine` for date/dependency math and emitting domain events (e.g., `task.updated`, `issue.sla_breached`).
3. **`{module}.repository.ts`:** Optimized database queries with cursor/offset pagination and bulk `INSERT / UPDATE` operations for sheet multi-select actions.
4. **`dto/*.dto.ts`:** Request/Response DTOs bound to `packages/shared-types` and OpenAPI/Swagger decorators.
5. **`{module}.spec.ts`:** Automated integration tests.

Layer 5: Real-Time & Background Worker Files (`apps/realtime` & `apps/worker`)

- **Real-Time (`apps/realtime`):** When User A edits a cell in the Task List Sheet (e.g., changing a task's status or end date), the API publishes a Redis Pub/Sub event. `project-sheet.room.ts` broadcasts a delta patch to all teammates viewing that project so their Task List Sheet, Kanban Board, or Gantt Chart updates in under 50ms without a page reload.
- **Workers (`apps/worker`):** Heavy or time-delayed operations (spawning recurring tasks at midnight, firing 9:00 AM task reminders, recalculating a 500-task Gantt dependency cascade, or dispatching external webhooks) are pushed to BullMQ queues and processed asynchronously.

Layer 6: Frontend Views & Virtualized Sheet/Gantt Components (`apps/web/src/*`)

- **URL-Driven State Architecture:** Every view state—active filter, sort order, grouping, and open slide-over drawer (`?view=sheet&filterId=active&taskId=8492`)—is synchronized with the URL so every screen state is shareable and bookmarkable.
- **Virtualized Rendering for the Task List Sheet:** A real-world project can contain 2,000+ tasks across 15 phases and 40 task lists. `useWbsTreeFlattener.ts` flattens the hierarchical tree (`Phase -> TaskList -> Task -> Subtask`) into a 1D virtualized array rendered via `@tanstack/react-virtual` and `@tanstack/react-table`, ensuring only the ~35 visible rows are mounted in the DOM at any time for buttery-smooth 60fps scrolling and instant keyboard cell editing.

7. Recommended Phased Execution Roadmap

To start generating the actual code files systematically without getting blocked:

1. **Step 1 (Foundation & Database):** Initialize the Turborepo workspace files (`package.json`, `turbo.json`, `docker-compose.yml`) and write the complete **Prisma/PostgreSQL schema files** (`packages/database`) covering Portals, Users/RBAC, Projects, Phases, Task Lists, Tasks/Subtasks, and Issues.

2. **Step 2 (Core Backend APIs):** Build the NestJS modules for `portals`, `projects`, `phases`, `task-lists`, and `tasks` (including the AST Filter compiler and WBS rollup calculations).
3. **Step 3 (The Task List Sheet & Slide-Over Drawer):** Build the primary Next.js workspace layout and the **Task List Sheet View** (collapsible Phases, Task Lists, inline cell editing, and the Task Detail Slide-Over Drawer).
4. **Step 4 (Kanban, Gantt, Issues & Timesheets):** Layer on the Kanban Board, Interactive Gantt Chart with dependencies, Issue Tracker, and Global Timer/Timesheets.
5. **Step 5 (Blueprints, EVM & Real-Time Collaboration):** Add the Visual Blueprint State Machine, EVM financial dashboards, and Yjs collaborative Pages.

Sources

1. Plane Review: Open-source Jira alternative for self-hosting teams
2. Best Project Management Software: 11 Tools Compared - Asana
3. Best Project Management Software in 2026: 15 Tools Compared
4. Taiga: Your opensource agile project management software
5. Plane: open-source project management with a surprisingly clean
6. Project Management Software Features - Zoho
7. Simple, Extensible, Open-Source Project Management Tool - Plane

---

User prompt: detailed backend design roadmap and hierarchy system to build this on Firebase. Also this whole project would be done on Antigravity + Firebase + Google AI Models. I need - 1. Backend Roadmap 2. Project completion roadmap for this whole Cloning System, How much time will be required for this Project. 3. I'm the Project manager for this, and this whole will be developed by a Developer using Antigravity.; Need a whole Task ledger for them to assign. In short, YOU are the SYSTEM Engineer, I'm the Project manager, The Developer and the Antigravity Agents will be the Dev Team. Lets lock in that way

Response: We are locked in. As your **Chief System Engineer**, I have architected this entire platform specifically for your stack: **Google Antigravity (Agentic IDE) + Firebase (BaaS) + Google AI Models (Gemini 2.5 Pro & Flash via Firebase Genkit).**

Moving from a traditional custom backend to **Firebase + Antigravity** is a massive force multiplier: Firebase eliminates ~65% of backend boilerplate (WebSocket plumbing, auth session servers, S3 multipart upload servers, and basic CRUD endpoints), allowing your Developer and their Antigravity Agents to focus entirely on **business logic, WBS hierarchy rules, the Task List Sheet UI, and AI automation.**

Here is your complete **System Engineering Blueprint**, structured into the three deliverables you requested:

1. **Firebase + Google AI Backend Architecture & Data Hierarchy System**
2. **Master Project Completion Roadmap & Time Estimate (12-Week Sprint Plan)**
3. **The Complete PM Task Ledger (72 Assignable Tickets with Antigravity Agent Specs)**

PART 1: Firebase + Google AI Backend Design & Hierarchy System

1.1 The Golden Architectural Rule on Firestore: Why Deep Subcollections Fail for Zoho Projects

In Zoho Projects, the conceptual hierarchy is 5 levels deep: `Portal -> Project -> Phase (Milestone) -> Task List -> Task -> Subtask`

The Trap: If your developer builds this as nested Firestore subcollections (`portals/{pId}/projects/{prjId}/phases/{phId}/taskLists/{tlId}/tasks/{tId}/subtasks/{stId}`), **the system will break** when you try to:

1. **Move a Task** from one Task List to another, or **move a Task List** from one Phase to another (in Firestore, changing a document's path requires deleting the document and recursively copying all its subcollections).
2. **Load the Task List Sheet or Gantt Chart** for an entire project in one query (you would have to fire dozens of chained reads across every phase and task list).
3. **Run Portal-Wide Queries** like *"My Work (All tasks assigned to me across 20 projects)"* or cross-project resource heatmaps.

1.2 The Solution: The **Portal-Scoped Flat Relational + Materialized Path Pattern**

To get **both** 1-query speed and effortless drag-and-drop reparenting, every major entity lives as a **direct subcollection under `portals/{portalId}`**, linked via indexed foreign keys (`projectId`, `phaseId`, `taskListId`, `parentTaskId`) and a **Materialized Path array** (`ancestorIds`).

```
FIRESTORE ROOT
└── portals/{portalId}                                  # Tenant root document (Settings, Business Hours, Holidays)
    ├── members/{userId}                                # Portal-scoped User Profile (ZPUID, Role, Cost Rate, Client Fla
    ├── teams/{teamId}                                  # Department / Group definitions
    ├── roles/{roleId}                                  # RBAC Permission Matrix
    ├── layouts/{layoutId}                              # Custom Field schemas & status workflows
    ├── blueprints/{blueprintId}                        # State-machine graphs, transition guards & post-actions
    ├── customViews/{viewId}                            # Saved AST Filter definitions (Personal / Project / Portal)
```

```
    ├── projectGroups/{groupId}                      # Portfolio groupings
    │
    ├── projects/{projectId}                         # Project metadata, roll-up counters, EVM summary, prefix counte
    ├── phases/{phaseId}                             # Indexed by: projectId, sequence, status, flag (internal/extern
    ├── taskLists/{taskListId}                       # Indexed by: projectId, phaseId, milestoneSequence, projectSequ
    ├── tasks/{taskId}                               # Tasks AND Subtasks (Indexed by: projectId, phaseId, taskListId
    ├── issues/{issueId}                             # Bugs (Indexed by: projectId, severity, statusId, affectedMiles
    │
    ├── timeLogs/{timeLogId}                         # Billable/Non-billable logs (Indexed by: projectId, taskId, use
    ├── expenses/{expenseId}                         # Non-labor project expenses
    ├── invoices/{invoiceId}                         # Generated client invoices
    │
    ├── files/{fileId}                               # File metadata, folder parentId, version history array, proofin
    ├── wikiPages/{pageId}                           # Project knowledge base pages
    ├── comments/{commentId}                         # Polymorphic comments (Indexed by: entityType, entityId, projec
    ├── notifications/{notificationId}               # User inbox alerts (Indexed by: recipientId, isRead, createdAt)
    └── auditLogs/{logId}                            # Immutable security & mutation trail
```

Why This Hierarchy Is Superior on Firebase:

- **1-Query Project Sheet Load:** To render the entire **Task List Sheet** or **Gantt Chart** for Project X, the frontend subscribes to just three parallel real-time `onSnapshot` listeners:
  1. `phases` where `projectId == X`
  2. `taskLists` where `projectId == X`
  3. `tasks` where `projectId == X` The frontend's `useWbsTreeFlattener` hook assembles the 5-level tree in memory in less than 4 milliseconds.
- **Zero-Cost Drag-and-Drop Reparenting:** Moving a Task to a different Task List or Phase is a single atomic `updateDoc(taskRef, { taskListId: newId, phaseId: newPhaseId })`.
- **Instant Subtask Rollups:** Every Subtask is simply a document in `tasks/{taskId}` with `depth: 1` (or higher), `parentTaskId: "task_abc"`, `rootTaskId: "task_root"`, and `ancestorIds: ["task_root", "task_abc"]`. Querying all nested subtasks at any depth under a root task is a single query: `where('ancestorIds', 'array-contains', rootTaskId)`.

1.3 Core Firestore Document Schemas (Production Contracts)

Your developer and Antigravity agents will implement these exact TypeScript/Firestore document shapes:

```
// 1. portals/{portalId}/projects/{projectId}
export interface ProjectDoc {
  id: string;
  portalId: string;
  name: string;
  keyPrefix: string;                // e.g., "ZPA" -> generates ZPA-1, ZPA-2
  taskSeqCounter: number;           // Atomic counter incremented via Firestore Transaction
  issueSeqCounter: number;
  ownerId: string;                  // userId (ZPUID)
  memberIds: string[];              // Used by Firestore Security Rules for O(1) access checks
  clientCompanyIds: string[];
  projectGroupId: string | null;
  layoutId: string;
  statusId: string;
  projectType: 'active' | 'template' | 'archived' | 'trashed';
  isPublic: boolean;
  isRollupProject: boolean;
  isStrictSchedule: boolean;        // Enforces task dates stay inside phase/project dates
  startDate: string;                // ISO YYYY-MM-DD
  endDate: string;
  budgetConfig: {
    currency: string;
    budgetType: 'none' | 'based_on_project' | 'based_on_milestone' | 'based_on_task' | 'based_on_user';
    billingMethod: 'project_hours' | 'staff_hours' | 'fixed_cost' | 'task_or_issue_hours';
    plannedCost: number;
    actualCost: number;             // Rolled up via Cloud Functions
    plannedHours: number;
    actualHours: number;
    revenueBudget: number;
    earnedValue: number;            // EVM metric
  };
  stats: {
    openTaskCount: number;
    closedTaskCount: number;
```

```
    openIssueCount: number;
    closedIssueCount: number;
    completionPercentage: number;
    healthScore: 'on_track' | 'at_risk' | 'delayed';
  };
  customFields: Record<string, any>;
  createdAt: number;
  updatedAt: number;
}

// 2. portals/{portalId}/taskLists/{taskListId}
export interface TaskListDoc {
  id: string;
  portalId: string;
  projectId: string;
  phaseId: string | null;           // null = "None / Unassigned Phase"
  name: string;
  flag: 'internal' | 'external';    // CRITICAL: Controls Client User visibility
  isGeneralDefault: boolean;        // Auto-created default list
  isCompleted: boolean;
  isRolledOver: boolean;
  sequence: {
    milestoneSequence: number;      // LexoRank / integer order inside the Phase
    projectSequence: number;        // Global order inside the Project Sheet
  };
  stats: {
    openTaskCount: number;
    closedTaskCount: number;
    completionPercentage: number;
  };
  createdAt: number;
  updatedAt: number;
}

// 3. portals/{portalId}/tasks/{taskId}
export interface TaskDoc {
  id: string;
  portalId: string;
  projectId: string;
  phaseId: string | null;
  taskListId: string;
  prefixKey: string;                // e.g., "ZPA-104"
  sequenceNumber: number;           // 104
  name: string;
  descriptionHtml: string;
  flag: 'internal' | 'external';    // Inherited from TaskList, enforced in Security Rules

  // Hierarchy (Subtasks)
  depth: number;                    // 0 = Top-level Task, 1 = Subtask, 2 = Sub-subtask
  parentTaskId: string | null;
  rootTaskId: string | null;
  ancestorIds: string[];            // Materialized path for instant tree queries
  sheetSequence: string;            // LexoRank string (e.g., "0|hzzzzz:") for O(1) drag-and-drop reordering

  // Status & Priority
  statusId: string;
  isClosedType: boolean;
  isCompleted: boolean;
  priority: 'none' | 'low' | 'medium' | 'high';
  completionPercentage: number;     // 0 - 100

  // Scheduling & Dependencies
  startDate: string | null;         // ISO timestamp or YYYY-MM-DD
  endDate: string | null;
  duration: { value: number; type: 'days' | 'hours' };
  predecessors: Array<{
    taskId: string;
    type: 'FS' | 'SS' | 'FF' | 'SF';
    lagDays: number;
  }>;
  successorIds: string[];           // Denormalized for fast Cloud Function cascade traversal
  isOnCriticalPath: boolean;
  baselineStartDate: string | null;
  baselineEndDate: string | null;
```

```
  // Ownership & Work Allocation (Zoho's owners_and_work model)
  ownerIds: string[];               // Flat array for Firestore `array-contains` queries
  ownersAndWork: {
    workType: 'standard' | 'flexible';
    unit: 'hours' | 'hours_per_day' | 'percent_per_day';
    copyTaskDuration: boolean;
    totalPlannedHours: number;
    allocations: Record<string, {   // Keyed by userId
      zpuid: string;
      standardValue?: string;       // e.g., "04:00" or "50"
      flexibleDailyMatrix?: Record<string, string>; // {"2026-10-20": "04:00"}
    }>;
  };

  // Financials & Time Tracking
  billingType: 'none' | 'billable' | 'non_billable';
  loggedHours: {
    billableMinutes: number;
    nonBillableMinutes: number;
    totalMinutes: number;
  };
  budget: {
    plannedHours: number;
    plannedCost: number;
    actualCost: number;
    revenueBudget: number;
  };

  // Automation, Checklists & Custom Fields
  blueprintId: string | null;
  checklistItems: Array<{ id: string; label: string; checked: boolean; assigneeId?: string }>;
  recurrence: {
    isActive: boolean;
    frequency: 'none' | 'daily' | 'weekly' | 'monthly' | 'yearly';
    interval: number;
    nextRecurrenceAfterClose: boolean;
    setToPreviousBusinessDay: boolean;
  } | null;
  tagIds: string[];
  followerIds: string[];
  customFields: Record<string, string | number | boolean | string[]>; // Indexed via wildcard/specific indexes
  embeddingVector?: number[];       // 768-dim Gemini text-embedding-004 vector for AI search
  isTrashed: boolean;
  createdBy: string;
  createdAt: number;
  updatedAt: number;
}
```

1.4 The 6-Service Firebase Backend Stack

To handle every Zoho Projects feature without hitting Firestore limitations, the backend splits duties across **6 native Firebase/Google Cloud services**:

Service	Exact Role in Our System	Why This Service?
1. Cloud Firestore	Primary persistence & real-time listener engine for Portals, Projects, Phases, Task Lists, Tasks, Issues, Comments, and Views.	Built-in offline IndexedDB cache + real-time delta push to the Task List Sheet, Kanban, and Gantt views.
2. Firebase Realtime Database (RTDB)	Ephemeral state: **Live User Presence**, **Active Cell Cursors** in the Task List Sheet, **Live Running Stopwatch Timers**, and **Yjs Collaborative Wiki awareness.**	Firestore charges per document write; RTDB charges by bandwidth and handles 10,000+ cursor/presence heartbeats per minute at near-zero cost and ~15ms latency.
3. Cloud Functions for Firebase (2nd Gen)	Serverless domain engines: Dependency DAG cascade solver, Blueprint state-machine transition validator, EVM financial rollup calculator, and AST filter API.	Runs on Cloud Run (Node 22 / TypeScript), supports up to 32GB RAM and concurrency (up to 1,000 requests/instance) with zero cold-start issues when `minInstances: 1` is set on core callable functions.
4. Cloud Tasks + Cloud Scheduler	Precision async queues: **Task Reminders** (fire at exact timestamp), **Issue SLA Breach Escalations**, **Midnight Recurring Task Spawner**, and **Webhook Retries.**	Unlike cron polling, Cloud Tasks lets you schedule a task to fire at the *exact minute* an Issue's 4-hour SLA expires, and cancel the task if the bug is resolved early.

Service	Exact Role in Our System	Why This Service?
5. Cloud Storage for Firebase	Project File Repository, multi-version asset storage (`v1`, `v2`, `v3`), and visual proofing attachments.	Direct browser-to-bucket resumable uploads secured by Firebase Storage Rules tied to Firestore project membership.
6. Firebase Genkit + Vertex AI (Gemini 2.5)	AI Co-Pilot flows: Natural-language-to-WBS generator, Risk/Bottleneck predictor, Comment thread summarizer, and Firestore Vector Search.	Native Firebase integration with structured Zod output schemas, streaming responses, and Firebase App Check protection.

1.5 Backend Execution Roadmap (7 Backend Phases)

1. **Backend Phase 1 — Multi-Tenant Auth, Custom Claims & Security Rules**
   - Configure Firebase Auth triggers (`beforeUserSignedIn`, `onCall(switchPortal)`) to inject custom JWT claims: `{ activePortalId, role: 'admin'|'manager'|'member'|'client', profileId }`.
   - Write `firestore.rules` that strictly block `role == 'client'` from reading any document where `resource.data.flag == 'internal'`.
2. **Backend Phase 2 — Atomic WBS Engine & Sequence Generator**
   - Build Callable Cloud Functions and Firestore Transactions for Project, Phase, and Task List creation, including atomic `taskSeqCounter` increments (`ZPA-101`, `ZPA-102`) and automatic creation of default `internal` and `external` Task Lists.
3. **Backend Phase 3 — Task Engine, DAG Dependency Solver & Work Allocation**
   - Implement the `onDocumentUpdated("portals/{portalId}/tasks/{taskId}")` trigger. When a task's `endDate` changes, traverse its `successorIds` Directed Acyclic Graph (DAG), apply portal `businessHours` and `holidays`, shift dependent tasks, and recalculate the Critical Path (`isOnCriticalPath`).
4. **Backend Phase 4 — AST Filter Engine & Custom Fields Query Service**
   - Build the `executeAstFilter` 2nd-Gen Callable Function that parses Zoho-style boolean patterns (`(1 AND 2) OR 3`) and relative date macros (`today`, `next_7_days`, `overdue`) into parallel Firestore queries (or BigQuery/Data Connect SQL for complex multi-OR analytics) and returns merged, paginated results.
5. **Backend Phase 5 — Blueprint State Machine & Issue SLA Engine**
   - Build `executeBlueprintTransition` Callable Function: validates **Before** guards (role/criteria), validates **During** mandatory payload fields, commits the status transition atomically, and executes **After** post-actions (webhooks, field updates, notifications).
   - Connect Issue creation to Google Cloud Tasks to enqueue exact SLA warning and breach timers.
6. **Backend Phase 6 — Timesheets, EVM Rollup Engine & File Versioning**
   - Build `onTimeLogWrite` and `onExpenseWrite` triggers that atomically roll up `billableMinutes`, `actualCost`, `earnedValue` (EV), `CPI`, and `SPI` up the chain: `Task -> TaskList -> Phase -> Project`.
7. **Backend Phase 7 — Firebase Genkit + Gemini 2.5 AI Flows**
   - Build 4 Genkit flows using **Gemini 2.5 Flash** (for low-latency inline thread summarization, smart task triage, and semantic vector search via `text-embedding-004`) and **Gemini 2.5 Pro** (for deep WBS project generation and critical-path bottleneck analysis).

PART 2: Project Completion Roadmap & Timeline Estimate

2.1 How Much Time Is Required? (Traditional vs. Developer + Antigravity)

Metric	Traditional Dev Team (4 Devs, Custom SQL/API)	**1 Developer + Antigravity Agents + Firebase**	Time Saved
Phase 1: Core WBS + Task List Sheet MVP	10 – 12 Weeks	**4 Weeks (Sprints 1–2)**	**~65% Faster**
Phase 2: Gantt, Kanban, Issues, Timesheets & Views	12 – 14 Weeks	**4 Weeks (Sprints 3–4)**	**~70% Faster**
Phase 3: Blueprints, EVM Finance, Files, Wiki & AI	10 – 12 Weeks	**4 Weeks (Sprints 5–6)**	**~65% Faster**
Total Time to Full Production Launch	**32 – 38 Weeks (~8.5 Months)**	**12 Weeks (3 Months / 60 Working Days)**	**~5.5 Months Saved**

Why 12 Weeks Is Realistic with Antigravity:

In **Google Antigravity**, your Developer does not type out 600 boilerplate files line-by-line. Instead, the Developer acts as the **Lead Orchestrator**:

1. They drop our **System Rules & Firestore Schema** into `.antigravity/rules.md`.
2. They spawn **parallel Antigravity Agent tasks** per ticket from the Ledger below—one agent writes the Cloud Function + Emulator unit tests in the terminal while a second agent scaffolds the Next.js UI component and verifies it in Antigravity's built-in headless browser.
3. The Developer spends 70% of their time reviewing agent PR diffs, testing edge cases in the **Firebase Local Emulator Suite**, and tuning UX polish.

2.2 The 12-Week Master Schedule (6 Two-Week Sprints)

> Weeks 1 – 2
> • **Sprint 1: Foundation, Multi-Tenant Portals, Auth & RBAC**
>
> Bootstrap Next.js + Firebase Monorepo, Local Emulator Suite, Antigravity rules, Multi-Portal Auth, Custom Claims, Role/Profile Permission Matrix, Business Hours/Holidays, and Client User isolation (`internal` vs. `external` Security Rules).

> Weeks 3 – 4
> • **Sprint 2: WBS Hierarchy & The Task List Sheet Engine (Core MVP)**
>
> Build Projects, Project Groups, Phases (Milestones), Task Lists, and Tasks/Subtasks. Deliver the signature **Virtualized Task List Sheet View** (inline cell editing, keyboard navigation, multi-owner work allocation, and the Task Detail Slide-Over Drawer). **Milestone Gate 1: Usable Internal Alpha.**

> Weeks 5 – 6
> • **Sprint 3: Kanban, Interactive Gantt Chart, Calendar & AST Filters**
>
> Deliver the Drag-and-Drop Kanban Board, Interactive Gantt Chart with SVG Dependency Lines (`FS/SS/FF/SF`), Critical Path solver, Cloud Function dependency date-cascade engine, Custom Fields, and the `(1 AND 2) OR 3` AST Filter & Custom Views engine.

> Weeks 7 – 8
> • **Sprint 4: Issue/Bug Tracker, SLA Queues, Timesheets & EVM Finance**
>
> Build the Issue/Bug module, Cloud Tasks SLA escalation engine, Global Header Live Stopwatch Timer, Weekly Timesheet Approval Grid, Rate Cards, Expenses, EVM Financial Dashboard (PV/EV/AC/CPI/SPI), and PDF Invoice generator. **Milestone Gate 2: Feature Parity Beta.**

> Weeks 9 – 10
> • **Sprint 5: Blueprint Studio, File Repository, Proofing & Real-Time Wiki**
>
> Deliver the ReactFlow Visual Blueprint State-Machine Editor, Workflow Business Rules, Firebase Storage File Repository with versioning, Image/PDF Visual Proofing pins, RTDB Live Sheet Cursors, and Yjs Real-Time Collaborative Wiki Pages.

> Weeks 11 – 12
> • **Sprint 6: Google AI (Genkit + Gemini 2.5), BI Reports, Importers & Launch**
>
> Integrate 4 Gemini 2.5 Genkit AI agents (AI WBS Generator, Risk Predictor, Thread Summarizer, Vector Semantic Search), Global Resource Utilization Heatmap, BI Report Builder, Zoho/Jira/CSV Importer, E2E Playwright Verification, and Firebase App Hosting production launch. **Milestone Gate 3: Production V1.0 Release.**

PART 3: The Complete PM Task Ledger (72 Tickets for Developer + Antigravity)

As Project Manager, copy this ledger directly into your tracker.

- **How to use the `Mode` column:**
  - 🤖 **`Antigravity Auto`**: The developer assigns this directly to an Antigravity Agent using the provided prompt spec; the agent writes the code, runs the emulator test, and presents the diff.
  - 🤝 **`Hybrid (Agent + Dev)`**: Antigravity generates the 80% implementation; the Human Developer refines complex UX interactions or security edge cases.
  - 👨‍💻 **`Human Dev Lead`**: Infrastructure credentials, architecture sign-off, or final security review performed directly by your Developer.

SPRINT 1: Foundation, Multi-Tenant Portals, Auth & RBAC (Weeks 1–2 | 80 Hours)

Ticket ID	Task Title	Target Files / Layer	Mode	Antigravity Agent Prompt / Technical Specification	Deps
ZP-001	Monorepo & Firebase Emulator Bootstrap	`package.json`, `firebase.json`, `.firebaserc`, `turbo.json`	Human Dev	Initialize pnpm Turborepo with `apps/web` (Next.js 15 App Router), `functions/` (Firebase Cloud Functions v2 TypeScript), and `packages/shared-types`. Configure Firebase Local Emulator Suite for Auth, Firestore, RTDB, Functions, Storage, and PubSub.	None
ZP-002	Antigravity Context & Rules Configuration	`.antigravity/rules.md`, `packages/shared-types/src/models.ts`	Hybrid	Create `.antigravity/rules.md` encoding the Portal-Scoped Flat Firestore hierarchy, TypeScript strictness, Zod validation rules, and Security Rule conventions. Generate all core TypeScript interfaces (`PortalDoc`, `ProjectDoc`, `PhaseDoc`, `TaskListDoc`, `TaskDoc`, `IssueDoc`).	ZP-001
ZP-003	Multi-Tenant Firebase Auth & Session Flow	`apps/web/src/features/auth/*`, `functions/src/auth/*`	Antigravity Auto	Build Login, Register, Magic Link, Google/GitHub OAuth, and 2FA screens using Firebase Auth. Create `beforeUserSignedIn` blocking function to inject `{ activePortalId, role, zpuid, isClientUser }` into custom JWT claims.	ZP-002
ZP-004	Portal Creation & Onboarding Wizard	`apps/web/src/app/(auth)/onboarding/*`, `functions/src/portals/createPortal.ts`	Antigravity Auto	Build 4-step Onboarding Wizard (Company Name, unique `portalSlug`, Timezone/Currency, Business Hours). Implement Callable Function `createPortal` that atomically creates `portals/{portalId}`, default roles (`Admin`, `Manager`, `Employee`, `Client`), and owner membership.	ZP-003
ZP-005	Firestore & Storage Security Rules Engine	`firestore.rules`, `storage.rules`, `tests/security-rules.spec.ts`	Hybrid	Write helper functions `isPortalMember(portalId)`, `hasRole(portalId, roles)`, `isClientAllowed(resource)`. Enforce that any user with `isClientUser == true` CANNOT read/write any Phase, TaskList, Task, Comment, or File where `flag == 'internal'`. Write `@firebase/rules-unit-testing` suite.	ZP-004
ZP-006	Portal Switcher & App Shell Layout	`apps/web/src/app/(portal)/[portalSlug]/layout.tsx`	Antigravity Auto	Build collapsible left sidebar, top global bar, Portal Switcher dropdown, active stopwatch slot, notification bell, and theme switcher (Light/Dark/High-Contrast).	ZP-004
ZP-007	Roles, Profiles & Field-Level RBAC Matrix	`apps/web/src/app/(portal)/[portalSlug]/admin/roles/*`, `functions/src/rbac/*`	Antigravity Auto	Build UI and Firestore schema for custom Roles & Security Profiles. Include module-level CRUD toggles and field-level masking (e.g., hiding `costPerHour` and `budgetConfig` from non-managers).	ZP-005
ZP-008	Users, Client Companies & Teams Directory	`apps/web/src/app/(portal)/[portalSlug]/admin/users/*`, `functions/src/users/*`	Antigravity Auto	Build User Invitation flow (email link with role/client company assignment), Teams CRUD (grouping users into Frontend, QA, etc.), and user deactivation.	ZP-007

Ticket ID	Task Title	Target Files / Layer	Mode	Antigravity Agent Prompt / Technical Specification	Deps
ZP-009	Business Hours, Holidays & Leave Calendar	`apps/web/src/app/(portal)/[portalSlug]/admin/portal-settings/*`, `packages/shared-types/src/date-math.ts`	Antigravity Auto	Build Portal Business Calendar UI (working days, hours/day, regional holidays) and User Leave/Vacation tracker. Implement pure TypeScript `addBusinessDays(startDate, duration, calendar)` utility with Vitest tests.	ZP-002
ZP-010	Audit Log Interceptor & Recycle Bin (Trash)	`functions/src/audit/*`, `apps/web/src/app/(portal)/[portalSlug]/admin/audit-logs/*`	Antigravity Auto	Create reusable Cloud Function wrapper that logs before/after JSON diffs into `portals/{portalId}/auditLogs` on mutations. Build Admin Audit Trail viewer and 30-day Soft-Delete Recycle Bin (`isTrashed: true`) with restore action.	ZP-005
ZP-011	Global Command Palette (`Cmd+K`)	`apps/web/src/components/command-palette.tsx`	Antigravity Auto	Implement keyboard-first `Cmd+K` modal supporting fuzzy jump to any Project, View, Admin Screen, or Quick Action ("Create Task", "Start Timer", "Switch Theme").	ZP-006

SPRINT 2: WBS Hierarchy & The Task List Sheet Engine (Weeks 3–4 | 88 Hours)

Ticket ID	Task Title	Target Files / Layer	Mode	Antigravity Age Prompt / Techn Specification
ZP-012	Project Groups & Project CRUD Service	`functions/src/projects/*`, `apps/web/src/app/(portal)/[portalSlug]/projects/*`	Antigravity Auto	Build `createPro` and `updatePro` Callable Functio When a project i created, atomica initialize `taskSeqCounte` `issueSeqCount` `0`, and auto-crea default Task List (`General Inter` and `General External`). Build Project Portfolio Directory (Card/ views).
ZP-013	Project Templates & Deep Cloner (`copy_from`)	`functions/src/projects/cloneProject.ts`	Hybrid	Build `clonePro` Cloud Function t deep-copies Pha Task Lists, Tasks Subtasks, Dependencies, a Custom Field La from a source Project/Templat shifting all relativ start/end dates b on the new proje `startDate`.
ZP-014	Phases (Milestones) Engine & Chaining	`functions/src/phases/*`, `apps/web/src/app/(portal)/[portalSlug]/projects/[projectId]/phases/*`	Antigravity Auto	Implement Phas CRUD with `flag` (`internal/exte` budget allocatio sequential phas chaining (`startAfterPha` + `offsetDays`). Phases Directo

Ticket ID	Task Title	Target Files / Layer	Mode	Antigravity Age Prompt / Techn Specification
				View with progre bars.
ZP-015	Task Lists Engine, Reordering & Rollover	`functions/src/taskLists/*`	Antigravity Auto	Implement Task CRUD associate `phaseId` (or `nul` Unassigned). Bu LexoRank seque reordering (`milestoneSequ` `projectSequen` and `rolloverTaskL` function to move tasks from a clos list to a target phase/list.
ZP-016	Core Task & Infinite Subtask Backend Service	`functions/src/tasks/createTask.ts`, `functions/src/tasks/updateTask.ts`	Hybrid	Implement atom Task creation us Firestore Transa on `projectRef` increment `taskSeqCounte` assign `prefixKe` (`ZPA-101`). Popu `depth`, `parentTaskId`, `rootTaskId`, an `ancestorIds` ar for subtasks.
ZP-017	Multi-Owner Work Allocation Engine (`owners_and_work`)	`packages/shared-types/src/work-allocation.ts`, `apps/web/src/features/task-drawer/WorkAllocationModal.tsx`	Hybrid	Implement Zoho `ownersAndWork` calculation engir supporting **Stan** (`hours`, `hours_per_day` `%_per_day`) and **Flexible** (daily h matrix per assig modes. Build the interactive Work Allocation Matrix Modal.
ZP-018	Virtualized Task List Sheet Grid (WBS Tree)	`apps/web/src/features/task-sheet/components/TaskSheetGrid.tsx`, `useWbsTreeFlattener.ts`	Hybrid	Build the primary Zoho-style **Task Sheet View** usin `@tanstack/rea` `table` + `@tanstack/rea` `virtual`. Flatter `Phase -> Task -> Task -> Su` into a collapsible virtualized grid w frozen columns, column resizing, summary footer
ZP-019	Sheet Inline Cell Editors & Excel Keyboard Nav	`apps/web/src/features/task-sheet/components/cells/*`, `useSheetKeyboardNav.ts`	Antigravity Auto	Build inline cell e for Status, Priori Multi-Owner Ava Start/End Date + Duration, `%` `Completion` slid Billing Type, and Add arrow-key c focus, `Enter` to `Tab` to advance,

Ticket ID	Task Title	Target Files / Layer	Mode	Antigravity Age Prompt / Techn Specification
				bottom `InlineQuickAd`
ZP-020	Bulk Selection Toolbar & Batch Mutations	`apps/web/src/features/task-sheet/components/SheetBulkActionBar.tsx`, `functions/src/tasks/bulkUpdateTasks.ts`	Antigravity Auto	Build multi-row checkbox selecti (including "Selec in Task List") and floating bulk acti to batch-update Status, Priority, Assignee, Dates List, or Delete us Firestore `WriteB` (chunked at 500
ZP-021	Task Detail Slide-Over Drawer & Checklists	`apps/web/src/features/task-drawer/TaskDetailDrawer.tsx`	Antigravity Auto	Build URL-synce `taskId=ZPA-10` right slide-over d containing Rich T Description (TipT Subtask list, Checklists, Work Allocation summ Followers, Attachments, an Activity Diff Histo tab.
ZP-022	Polymorphic Threaded Comments & `@Mentions`	`apps/web/src/features/collaboration/CommentThread.tsx`, `functions/src/comments/*`	Antigravity Auto	Build rich-text comment system Tasks, Task Lists Phases, and Issu supporting `@use` autocomplete, `#` `101` live task pre chips, emoji reac and an `interna` `external` comm visibility toggle.

SPRINT 3: Kanban, Interactive Gantt, Dependencies & AST Filters (Weeks 5–6 | 86 Hours)

Ticket ID	Task Title	Target Files / Layer	Mode	Antigravity A Technical Sp
ZP-023	DAG Dependency Validator & Auto-Cascade Engine	`functions/src/tasks/dependencyCascade.ts`, `packages/shared-types/src/critical-path.ts`	Hybrid	Build topologi detector (bloc like `A -> B -` Firestore trigg predecessor cascade new `startDate/en` transitive suc `FF`, `SF` + `lagD` portal busines compute `isOn`
ZP-024	Interactive Gantt Chart Canvas & SVG Links	`apps/web/src/features/gantt-chart/*`	Hybrid	Build split-pan (WBS tree on timeline canv Day/Week/Mo zoom, drag-to bars, drag-ed duration, drag dependency a Path red high Baseline ghos
ZP-025	Kanban Board View with	`apps/web/src/features/kanban-board/*`	Antigravity Auto	Build drag-an Board using @ Support dyna

Ticket ID	Task Title	Target Files / Layer	Mode	Antigravity A Technical Sp
	Swimlanes & WIP Limits			grouping (by Assignee, or and horizonta Phase, Task l with LexoRan and column V warnings.
ZP-026	Project Calendar View & Unscheduled Tray	`apps/web/src/app/(portal)/[portalSlug]/projects/[projectId]/tasks/calendar/*`	Antigravity Auto	Build Month/V Calendar Viev Tasks, Phase Include a righ "Unscheduled be dragged o dates to set s `endDate`.
ZP-027	Custom Fields & Project Layout Designer	`apps/web/src/app/(portal)/[portalSlug]/automation/layouts/*`, `functions/src/customFields/*`	Antigravity Auto	Build Layout Designer sup Number, Curr Picklist, Multi- Checkbox, ar (`Estimated` Render custo dynamically a Task List She Task Drawer.
ZP-028	AST Boolean Filter Compiler (`(1 AND 2) OR 3`)	`packages/shared-types/src/ast-filter.ts`, `functions/src/views/executeAstFilter.ts`	Hybrid	Build parser a Zoho-style filt ({"criteria "pattern": 3"}) supporti relative date m `this_week`, m `overdue`, uns user macros (`current_lo`
ZP-029	Visual Filter Builder UI & Saved Custom Views	`apps/web/src/features/filter-custom-views/*`	Antigravity Auto	Build visual fi modal allowin criteria, edit b strings, config column sortin configurations Project, or Po Views.
ZP-030	Recurring Tasks Engine (Cloud Scheduler)	`functions/src/tasks/recurrenceWorker.ts`, `apps/web/src/features/task-drawer/RecurrenceReminderPanel.tsx`	Antigravity Auto	Build Recurre Cloud Schedu engine suppo `weekly`, `mont` `yearly` rules, `nextRecurre` `setToPrevio` and optional subtask/comm
ZP-031	Smart Task Reminders via Google Cloud Tasks	`functions/src/tasks/reminderScheduler.ts`	Antigravity Auto	Implement re scheduling (d date, `before/` end date). En execution job Cloud Tasks t Email (via Se extension) an notifications t or reporting m

SPRINT 4: Issues/Bugs, SLAs, Timesheets & EVM Finance (Weeks 7–8 | 84 Hours)

Ticket ID	Task Title	Target Files / Layer	Mode	Antigravi Technical
ZP-032	Issue / Bug Tracker Module & Drawer	`functions/src/issues/*`, `apps/web/src/app/(portal)/[portalSlug]/projects/[projectId]/issues/*`	Antigravity Auto	Build com with atomi (`BUG-1`, `BU Reproduc Classifi affected releaseM` Issue She View, and Drawer.
ZP-033	Issue-to-Task Conversion & Release Notes	`functions/src/issues/convertIssue.ts`	Antigravity Auto	Build 1-cli Task" and flows. Buil Notes Ger closed iss releaseM formatted
ZP-034	SLA Escalation Engine (Cloud Tasks)	`functions/src/issues/slaEngine.ts`, `apps/web/src/app/(portal)/[portalSlug]/projects/[projectId]/issues/slas/page.tsx`	Hybrid	Build SLA "Critical B Resolve ir Hours"). E issue crea countdow cards; trig escalation
ZP-035	Web-to-Issue Intake Form & Git Webhook Sync	`functions/src/integrations/gitWebhooks.ts`, `apps/web/src/app/public/forms/[formId]/page.tsx`	Antigravity Auto	Build shar submissio GitHub/Gi that parse messages or `Refs #` commit m and auto-t
ZP-036	Global Live Stopwatch Timer (Firebase RTDB)	`apps/web/src/stores/useGlobalTimerStore.ts`, `functions/src/timesheets/timerSync.ts`	Antigravity Auto	Build pers Timer bac (`timers/` Starting a across all stopping it confirmati `timeLogs`
ZP-037	Weekly Timesheet Grid & Approval Workflow	`apps/web/src/app/(portal)/[portalSlug]/timesheets/page.tsx`, `functions/src/timesheets/*`	Antigravity Auto	Build spre Timeshee entry acro tasks/issu Approval (`submitte rejected` and date-l
ZP-038	Rate Cards, Budgets & EVM Rollup Engine	`functions/src/budgets/evmRollup.ts`, `packages/shared-types/src/evm-math.ts`	Hybrid	Implemen `timeLogs` to calculat `* userCo actualRe * ratePe` (PV), Earn (`EV/AC`), a Phase, an

Ticket ID	Task Title	Target Files / Layer	Mode	Antigravi Technica
ZP-039	Project Finance Dashboard, Expenses & Invoices	`apps/web/src/app/(portal)/[portalSlug]/projects/[projectId]/finance/page.tsx`	Antigravity Auto	Build Proj featuring F Budget vs Expense I Threshold (80%/100 PDF gene billable log
ZP-040	"My Work" Personal Agenda & Global Feed	`apps/web/src/app/(portal)/[portalSlug]/my-work/page.tsx`, `feed/page.tsx`	Antigravity Auto	Build cros screen (gr Overdue, Unschedu Pending A company- Stream &
ZP-041	Roll-Up Projects & Portfolio Roadmap View	`apps/web/src/app/(portal)/[portalSlug]/projects/groups/page.tsx`	Antigravity Auto	Build Port and Roll-U that rolls u timelines, EVM budg master vie

SPRINT 5: Blueprint Studio, File Repository, Proofing & Real-Time Wiki (Weeks 9–10 | 84 Hours)

Ticket ID	Task Title	Target Files / Layer	Mode	Antigravity Ager Prompt / Techni Specification
ZP-042	Visual Blueprint State-Machine Editor (ReactFlow)	`apps/web/src/features/blueprint-studio/*`	Hybrid	Build drag-and-dr Visual Blueprint C using `@xyflow/r` (ReactFlow). Allo admins to place S Nodes, draw Tran Edges, and confi **Before** (allowed roles/criteria), **Du** (mandatory form fields/attachment and **After** (field u webhooks, alerts
ZP-043	Blueprint Runtime & Transition Prompt Modal	`functions/src/blueprints/executeTransition.ts`, `apps/web/src/features/task-drawer/BlueprintTransitionBar.tsx`	Hybrid	Replace standard dropdown on Blu enabled tasks wit Transition Button Clicking a transiti opens the dynam **During** validation calls `executeTransit` Cloud Function, e guards, and runs actions.
ZP-044	Event-Driven Business Rules & Outgoing Webhooks	`functions/src/automation/businessRulesEngine.ts`, `webhookDispatcher.ts`	Antigravity Auto	Build `On Create`, `Field Update`, a `Overdue` Busines engine and Outg Webhook dispatc (with HMAC-SHA signature header Cloud Tasks expo backoff retries).

Ticket ID	Task Title	Target Files / Layer	Mode	Antigravity Age Prompt / Techni Specification
ZP-045	Project File Repository & Version Control	`apps/web/src/features/file-vault/*`, `functions/src/files/*`	Antigravity Auto	Build hierarchical Tree and File Gri backed by Cloud for Firebase. Sup drag-and-drop up `internal/exter` visibility flags, Ch In/Check-Out locl and uploading ne versions (`v1`, `v2`, rollback.
ZP-046	Visual Proofing & Asset Annotation Lightbox	`apps/web/src/features/file-vault/ProofingLightbox.tsx`	Hybrid	Build full-screen I PDF Proofing Vie where reviewers coordinates (`x%`, draw bounding bo a design mockup numbered feedba threads (`#1`, `#2`) a mark files as "Ap or "Changes Req
ZP-047	Real-Time Collaborative Wiki Pages (TipTap + Yjs)	`apps/web/src/features/wiki-pages/*`, `functions/src/wiki/*`	Hybrid	Build nested Proj Pages / Wiki tree Notion-style TipTa editor powered by Firebase RTDB p for live multi-user cursors, slash commands, embe live Task List tabl version snapshot
ZP-048	Live Presence & Sheet Cell Locking (RTDB)	`apps/web/src/features/task-sheet/hooks/useSheetPresence.ts`	Antigravity Auto	Use Firebase RT broadcast which teammates are c viewing a Project or Task Drawer, a highlight the activ border with the teammate's name when they are inl editing a cell.
ZP-049	Project Discussion Forums & Live Chat Rooms	`apps/web/src/app/(portal)/[portalSlug]/projects/[projectId]/discussions/*`	Antigravity Auto	Build categorized form Discussion I and real-time Pro Chat channels (w sharing, `@mentio` "Convert chat me to Task" action).
ZP-050	Notification Center (In-App, Email Digest & Push)	`apps/web/src/features/notifications/*`, `functions/src/notifications/*`	Antigravity Auto	Build slide-over Notification Inbox Unread, `@Mentio` Assigned to Me, Approvals) with F Cloud Messaging web push and configurable daily digests.

SPRINT 6: Google AI (Genkit + Gemini 2.5), BI Reports, Importers & Launch (Weeks 11–12 | 86 Hours)

Ticket ID	Task Title	Target Files / Layer	Mode	Antigravity Agent Prompt / Technical Specification	D
ZP-051	Firebase Genkit + Gemini 2.5 Pro WBS Co-Pilot	`functions/src/ai/generateProjectWbs.ts`, `apps/web/src/features/ai-copilot/AiWbsModal.tsx`	Hybrid	Build a Firebase Genkit flow using `gemini-2.5-pro` with structured Zod output schema. Given a natural language product prompt (or uploaded PRD PDF), generate a complete WBS (`Phases -> Task Lists -> Tasks -> Subtasks`, durations, dependencies, and role estimates) with an interactive preview before committing to Firestore.	
ZP-052	Gemini 2.5 Risk & Schedule Bottleneck Predictor	`functions/src/ai/analyzeProjectRisk.ts`	Antigravity Auto	Build Genkit flow that feeds the project's Critical Path, overdue tasks, assignee workload overload, and EVM burn rate into `gemini-2.5-pro` to output actionable schedule risk warnings and recommended date/owner reallocations.	
ZP-053	Gemini 2.5 Flash Thread Summarizer & Bug Triage	`functions/src/ai/summarizeAndTriage.ts`	Antigravity Auto	Build fast `gemini-2.5-flash` flows to: (1) summarize 50+ comments on a Task/Issue into key decisions & action items, and (2) auto-suggest `Severity`, `Classification`, potential duplicate bugs, and likely owner when a new Issue is created.	
ZP-054	Firestore Vector Search (`text-embedding-004`)	`functions/src/ai/vectorSearch.ts`	Antigravity Auto	Configure Firestore Vector Index (`768` dimensions) using Google's `text-embedding-004` model on Task, Issue, and Wiki Page writes. Add natural-language semantic search to the `Cmd+K` Command Palette (`findNearest`).	
ZP-055	Global Resource Utilization & Workload Heatmap	`apps/web/src/app/(portal)/[portalSlug]/workload/page.tsx`	Hybrid	Build interactive Resource Utilization Heatmap grid showing every team member's daily/weekly allocated hours (`ownersAndWork`) vs. available capacity (`businessHours - leaves`). Allow drag-and-drop task reassignment directly from the heatmap to balance overloaded (red) teammates.	
ZP-056	BI Analytics Hub & Custom Report Builder	`apps/web/src/app/(portal)/[portalSlug]/reports/*`	Antigravity Auto	Build Burndown/Burnup charts, Planned vs. Actual Hours, Milestone Slippage, Issue Aging/MTTR, and a Custom Pivot/Chart Report Builder with CSV/PDF export.	

Ticket ID	Task Title	Target Files / Layer	Mode	Antigravity Agent Prompt / Technical Specification	D
ZP-057	Zoho Projects / Jira / CSV / MPP Import & Export Wizard	`functions/src/integrations/importer.ts`, `apps/web/src/features/import-export/*`	🤖 Antigravity Auto	Build Import Wizard with column mapping for Zoho Projects JSON/CSV, Jira CSV, Asana CSV, and MS Project XML, plus full Task List Sheet & Gantt exports to `.xlsx`, `.csv`, and `.pdf`.	Z (0
ZP-058	Public Read-Only View Sharing & iCal Sync	`functions/src/views/publicShare.ts`	🤖 Antigravity Auto	Allow Project Managers to generate revocable, optionally password-protected external share links for a Gantt Chart or Kanban Board, plus two-way Google/Outlook Calendar iCal feed URLs.	Z (Z (0
ZP-059	Public REST API, Personal Access Tokens & OpenAPI Docs	`functions/src/publicApi/*`	🤖 Antigravity Auto	Expose rate-limited `/api/v1/portals/:portalId/...` REST endpoints authenticated via Personal Access Tokens (PATs) or OAuth2, complete with auto-generated Swagger/OpenAPI interactive documentation.	Z (0
ZP-060	E2E Playwright Test Suite & Firebase App Hosting Launch	`e2e/*`, `apphosting.yaml`, `firestore.indexes.json`	🤍 Hybrid	Run Antigravity Browser Agent + Playwright E2E test suite across all 12 modules. Deploy composite indexes (`firestore.indexes.json`), Cloud Functions v2, Storage Rules, and Next.js frontend to Firebase App Hosting.	A

PART 4: Your Day-1 PM Action Protocol (How to Run the Dev + Antigravity Team)

To manage your Developer and their Antigravity Agents with maximum velocity, enforce these **4 Project Management Rules** starting on Day 1:

1. **Lock the Schema Contract First (Tickets `ZP-001` & `ZP-002`):** Do not let the developer build UI screens before `packages/shared-types/src/models.ts` and `.antigravity/rules.md` are committed. When Antigravity agents have the exact TypeScript interfaces in context, they generate zero-hallucination code across both frontend and Cloud Functions.
2. **Enforce Emulator-First Verification:** Every ticket in the ledger above must be verified locally against the **Firebase Local Emulator Suite** (`firebase emulators:start`) before merging. Antigravity's terminal agent can automatically run `pnpm test:emulator` after writing any Cloud Function or Security Rule.
3. **Use Parallel Agent Workspaces Starting in Sprint 2:** Once Sprint 1 is complete, have your Developer run **two parallel Antigravity agent streams**:
   - *Agent Stream A (Backend/Functions):* Works on Cloud Functions, Firestore triggers, and unit tests.
   - *Agent Stream B (Frontend/Views):* Works on Next.js App Router views, Task List Sheet cells, and slide-over drawers bound to the shared TypeScript interfaces.
4. **Weekly Milestone Demo Gates:**
   - **End of Week 2:** Live demo of Multi-Portal Onboarding, Role Matrix, and Client User Security Rules blocking `internal` items.
   - **End of Week 4:** Live demo of creating a 5-level WBS project and inline-editing 100 tasks in the **Task List Sheet View**.

- **End of Week 6:** Live demo of the **Interactive Gantt Chart** auto-cascading dependent tasks and the `(1 AND 2) OR 3` AST filter bar.
- **End of Week 8:** Live demo of the **Issue Tracker**, SLA breach timers, Live Stopwatch, and EVM Financial Dashboard.
- **End of Week 10:** Live demo of the **Visual Blueprint Editor**, File Proofing pins, and Real-Time Wiki.
- **End of Week 12:** Live demo of **Gemini 2.5 AI WBS Co-Pilot**, Resource Heatmap, and production deployment.
